

알고리즘 중간고사 정리

이 과목을 한 줄로

"문제를 분류하고, 전략을 선택하고, 왜 그 전략이 빠른지 설명한다"

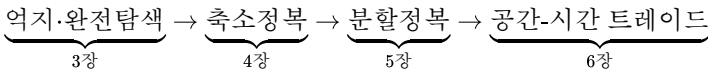
3장(억지) → 4장(축소정복) → 5장(분할정복) → 6장(공간으로 시간벌기)는 점점 똑똑해지는 전략의 계단이다.

0. 한눈에 보는 시험 핵심 지도

시험장에서 즉시 떠올라야 할 7가지

1. 복잡도 분석 순서 = 입력 크기 → 기본 연산 → 실행 횟수 → 점근 표기
2. 복잡도 위계 $1 < \log n < n < n \log n < n^2 < 2^n < n!$ — 절대 헛갈리지 말 것
3. 축소정복=작은 문제 1개 / 분할정복=여러 개 / 공간으로 시간벌기=전처리 테이블
4. 이진 탐색의 전제: 정렬 필수 · 해싱의 전제: 좋은 해시 함수
5. 분할정복 점근 복잡도 암기: 병합·퀵평균 $\Theta(n \log n)$ / 이진탐색·빠른 거듭제곱 $\Theta(\log n)$ / 스트라센 $\Theta(n^{2.807})$ (마스터 정리 유도
는 시험 범위 아님)
6. 해싱 평균 탐색 $\approx 1 + \alpha$ ($\alpha = n/m$)
7. DFS/BFS 둘 다 인접 리스트면 $O(V + E)$, 인접 행렬이면 $O(V^2)$

전략 계단 (중간고사 범위의 흐름)



원본 슬라이드

[Ch1 개요](#) · [Ch2 효율성](#) · [Ch3 억지](#) · [Ch4 축소정복](#) · [Ch5 분할정복](#) · [Ch6 공간-시간](#)

0-α. 비전공자용 용어 미니 사전

먼저 이 단어들부터 "일상어"로 알고 가자

용어	한 줄 뜻	일상 비유
알고리즘	문제 푸는 단계적 절차	라면 끓이는 레시피
복잡도 complexity	입력이 커질 때 시간·메모리가 얼마나 늘어나는가	손님 10명 vs 1000명일 때 설거지 시간
입력 크기 n	처리해야 할 데이터의 개수	책장에 꽂힌 책 수
기본 연산	코드에서 가장 자주 반복되는 한 스텝	비교 1회, 덧셈 1회
점근 표기 Asymptotic	아주 큰 입력에서의 속도 등급 표기	S·A·B·C 등급표
$O(n^2)$	"입력이 10배면 시간은 100배" 수준	운동장 구석구석 빠짐없이 훑기
$O(\log n)$	"입력이 10배여도 시간은 3배쯤"	스무고개로 답 좁히기
재귀 recursion	함수가 자기 자신을 다시 호출	러시아 인형(마트로시카)
그래프 graph	점(정점)과 점을 잇는 선(간선)의 모임	지하철 노선도
DAG	방향이 있고 순환이 없는 그래프	대학 커리큘럼(선수와목 → 본과목)

용어	한 줄 뜻	일상 비유
피벗 pivot	기준점으로 잡은 값	반 학생을 "내 키보다 작은/큰" 으로 가르는 나
해싱 hashing	이름을 숫자 주소로 바꿔 즉시 접근	사물함 번호표
DP / 동적 계획법	한 번 푼 부분 문제를 표에 저장	암기 노트

🔗 이 과목의 중심 질문 — 단 3개

1. 얼마나 빠르냐? → 복잡도 (2장)
2. 어떤 전략을 썼냐? → 억지/축소/분할/공간-시간 (3~6장)
3. 왜 그 전략이 그렇게 빠르거나 느린가? → 점화식 결과 (분할정복 점근 복잡도는 암기)

1. 알고리즘이란 무엇인가 (Ch1)

1-1. 정의

알고리즘 = 어떤 문제를 해결하는 명확한 단계적 절차

- 프로그래밍 언어 ≠ 알고리즘
- 같은 문제에 여러 알고리즘이 공존할 수 있음

예시: 최대공약수(GCD) 구하는 3가지 방법

1. 정의 그대로
2. 작은 수의 약수 열거
3. 유클리드 알고리즘

Ch.1 예제 — 유클리드 호제법 (Euclidean GCD): gcd(60, 24)

핵심 정리: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, $\text{gcd}(a, 0) = a \rightarrow$ 나머지가 0이 되는 순간 종료

① 세 가지 방법 비교 — 유클리드가 압도적

연속 정수 검사 (억지)	소인수분해 공통교집합	유클리드 호제법 ✓
$t = \min(a, b)$ while $a \% t \neq 0$ or $b \% t \neq 0$: $t \leftarrow t - 1$ $O(\min(a, b))$ — 느림	$60 = 2^2 \cdot 3 \cdot 5$ $24 = 2^3 \cdot 3$ 공통 = $2^2 \cdot 3 = 12$ 정수 분해 필요 — 어려움	while $b \neq 0$: $a, b \leftarrow b, a \bmod b$ return a $O(\log(\min(a, b)))$ — 최적

② gcd(60, 24) 단계별 실행 추적

단계	a	b	a mod b	조건 $b \neq 0$?	다음 ($a \leftarrow b, b \leftarrow a \bmod b$)
0 (초기)	60	24	$60 \bmod 24 = 12$	$24 \neq 0 \rightarrow$ 계속	(24, 12)
1	24	12	$24 \bmod 12 = 0$	$12 \neq 0 \rightarrow$ 계속	(12, 0)
2 (종료)	12	0	—	$b = 0 \rightarrow \text{STOP}$	return $a = 12$

③ 정당성: 왜 $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ 인가?

핵심: $a = qb + r$ (단, $r = a \bmod b$)
• a, b의 공약수 $d \rightarrow d \mid a, d \mid b \rightarrow d \mid (a - qb) = r \Rightarrow d$ 는 b, r의 공약수
• b, r의 공약수 $d \rightarrow d \mid b, d \mid r \rightarrow d \mid (qb + r) = a \Rightarrow d$ 는 a, b의 공약수
 $\therefore$ 두 공약수 집합이 동일 $\Rightarrow$ 최대공약수도 동일 $\Rightarrow \text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$

④ 시간 복잡도 — 감소 속도가 로그

Lamé 정리: 호출 횟수 $\leq 5 \cdot \log_{10}(\min(a, b))$ (피보나치 수가 최악 입력)
관찰: 두 단계마다 작은 값이 최소 절반 이하로 줄어듦 $\rightarrow T(n) = O(\log n)$
체크: $\text{gcd}(60, 24) \rightarrow$ 2단계 만에 종료. 억지(24회 시도)보다 12배 빠름

1-2. 좋은 알고리즘의 3조건

성질	뜻
정확성 correctness	유효한 모든 입력에 대해 정답 — "대부분 맞음"은 부족
효율성 efficiency	시간·공간을 적게 씀
일반성 generality	특수 입력만이 아닌 일반적 문제에도 동작

1-3. 과목 전체 로드맵

🔗 한 문제의 풀이 전략이 점점 똑똑해지는 순서로 배운다

3장(역지) → 4장(축소) → 5장(분할) → 6장(공간-시간)

1-4. 알고리즘 설계·분석 6단계 (PDF Ch1)

🔗 교재 표준 순서

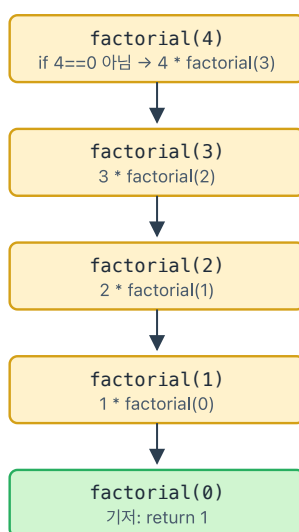
1. 문제 이해 (입출력 명세)
2. 계산 모델·자료구조 선택
3. 설계 전략 선택 (brute-force / decrease / divide / transform / DP / greedy)
4. 정확성 증명 (귀납법·불변식)
5. 효율성 분석 (시간·공간 복잡도)
6. 코딩·실험

1-5. 첫 재귀 예 — 팩토리얼

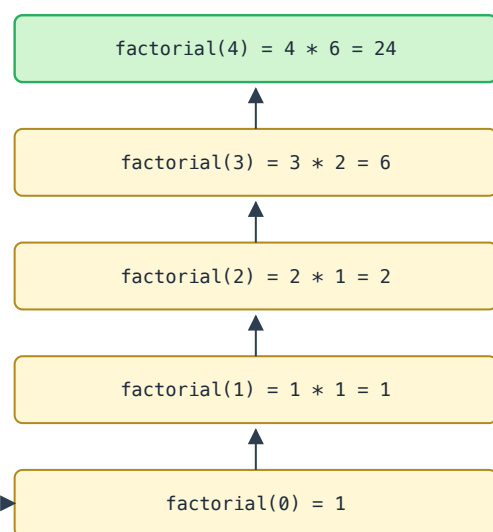
$n! = n \cdot (n-1)!$, $0! = 1$ — 자기 자신을 더 작은 입력으로 다시 호출. 내려갈 때 호출, 올라올 때 반환.

팩토리얼 재귀 호출 트리 ($n=4$) — 내려갈 때 호출, 올라올 때 반환

호출(다운):



반환(업):



점화식: $T(n) = T(n-1) + 1 \Rightarrow \Theta(n)$ · 기저: $\text{factorial}(0) = 1$

- 점화식: $T(n) = T(n-1) + 1 \Rightarrow \Theta(n)$
- 기저조건(base case) $\text{factorial}(0) = 1$ 없으면 무한 재귀

2. 효율성 분석 (Ch2)

2-1. 왜 이론적 복잡도인가

실제 실행 시간은 CPU·언어·컴파일러·입력에 따라 다름. 그래서 **입력 크기**에 대한 **기본 연산 실행 횟수**라는 이론적 척도를 씀.

≡ 분석의 4단계 — 순차합 1+2+...+n

입력 크기 → 기본 연산 식별 → 실행 횟수 Σ → 점근 표기

순차합 1+2+...+n 효율성 분석: 기본연산 세기 → Σ 전개 → $\Theta(n)$

① 의사코드

```
sum(n):  
  s ← 0  
  for i ← 1 to n do  
    s ← s + i ← 기본연산  
  return s
```

입력 크기: $n \cdot$ 기본연산: 덧셈 1회

→

② 실행 횟수 세기

```
i = 1 → 덧셈 1회  
i = 2 → 덧셈 1회  
i = 3 → 덧셈 1회  
...  
i = n → 덧셈 1회
```

총 덧셈 횟수 $C(n) = n$

③ Σ 표기로 전개

$$C(n) = \sum_{i=1..n} 1 = 1 + 1 + \dots + 1 \text{ (n번)} = n$$

※ 만약 본문이 "s ← s + i"의 덧셈+대입을 각각 세면 계수가 2가 되지만 점근적으로 동일

④ 점근 표기 유도

상한: $C(n) = n \leq 1 \cdot n$ 하한: $C(n) = n \geq 1 \cdot n \Rightarrow C(n) \in \Theta(n)$

결론: 순차합은 선형 시간, 입력 n이 10배 커지면 시간도 약 10배 증가.

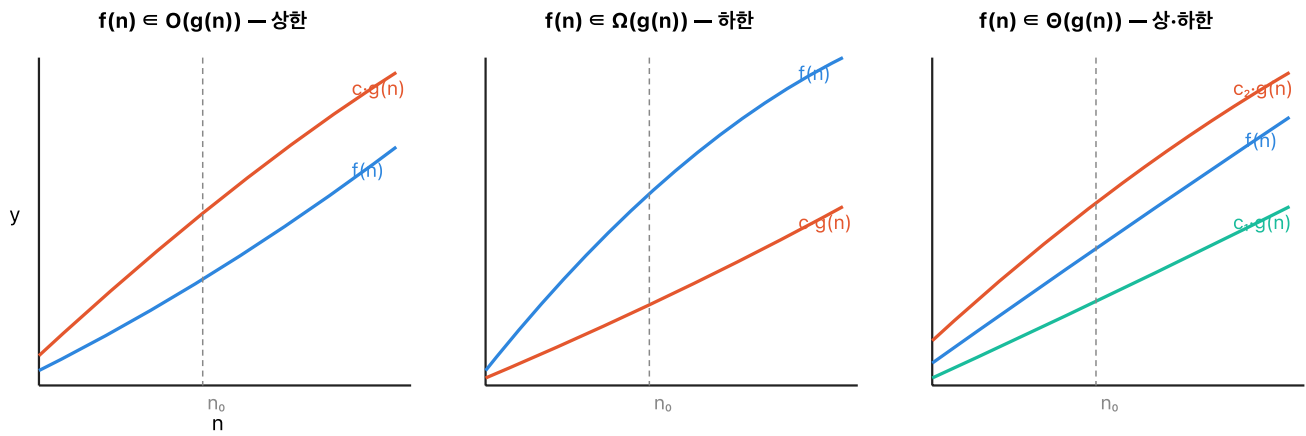
2-2. 최선 · 평균 · 최악

✍ 비전공자 비유 — 책 찾기

- **최선**: 맨 처음 잡은 책이 답
- **최악**: 마지막까지 못 찾음
- **평균**: 보통 절반쯤에서 찾는다고 가정

알고리즘	입력 상태 의존성
선택 정렬	무관 (항상 $\Theta(n^2)$)
삽입 정렬	거의 정렬되면 $O(n)$

2-3. 점근 표기 O, Ω, Θ



🕒 한마디로 뭘 하는 기호인가?

"입력이 충분히 커지면 이 알고리즘 속도는 대충 어느 등급에 속하는가?"를 표시하는 기호 3종 세트.

기호	뜻	일상 비유
$O(g)$	"아무리 나빠도 g 보다 빠르다" (상한·천장)	"시험 점수 최대 90점 이하"
$\Omega(g)$	"아무리 잘해도 g 보다 느리다" (하한·바닥)	"시험 점수 최소 60점 이상"
$\Theta(g)$	천장과 바닥이 같은 등급 (정확한 등급)	"딱 70~90점대 성적"

O = 위로 가두는 선 · Ω = 아래로 가두는 선 · Θ = 둘이 같은 차수로 겹칠 때

$$f \in O(g) \iff \exists c > 0, n_0 : 0 \leq f(n) \leq c g(n), \forall n \geq n_0$$

$$f \in \Omega(g) \iff \exists c > 0, n_0 : 0 \leq c g(n) \leq f(n), \forall n \geq n_0$$

$$f \in \Theta(g) \iff \exists c_1, c_2 > 0, n_0 : c_1 g(n) \leq f(n) \leq c_2 g(n)$$

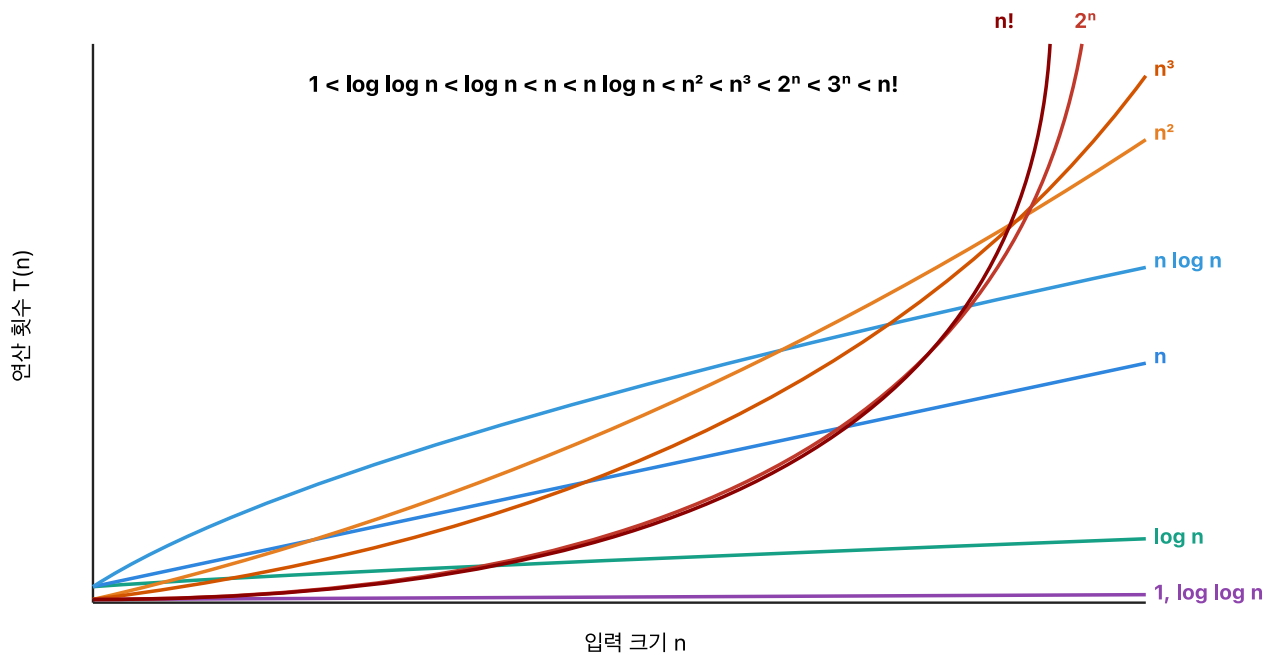
🔗 수식 해독 가이드 (기호에 겁먹지 말기)

- $\exists c > 0 \rightarrow$ "어떤 상수 c 를 찾을 수 있다"
- $n_0 \rightarrow$ "입력이 이 값 이상일 때부터" (작은 입력은 무시)
- $c g(n) \rightarrow g(n)$ 에 c 배를 곱한 선
- 전체 번역: " n 이 커지면 우리 함수는 g 의 상수배 안에 들어간다"

🔗 아이폰슬 필기 복원 (p.14)

$\alpha < \beta < \gamma$ 탐색 형광 $\rightarrow \Omega(n^\alpha) \leq \Theta(n^\beta) \leq O(n^\gamma)$ 하한-정확-상한 위계 감각

2-4. 복잡도 위계 — 반드시 암기



$$1 < \log \log n < \log n < n < n \log n < n^2 < n^3 < 2^n < 3^n < n! < n^n$$

빈출 비교:

- $n \log n < n^2$
- 모든 다항식 $n^k < \text{지수 } c^n$ (충분히 큰 n)
- $\log(n!) = \Theta(n \log n)$

🔗 $\log(n!)$ 유도 (p.22-24)

상한: $\log(n!) = \sum_{k=1}^n \log k \leq n \log n$

하한: $\log(n!) \geq \sum_{k=n/2}^n \log k \geq \frac{n}{2} \log \frac{n}{2} = \Omega(n \log n)$

2-5. 다단계 합성

$$T(n) = O(n \log n) + O(n) = \boxed{O(n \log n)}$$

가장 큰 항이 전체를 지배

🔗 합성 규칙 (PDF Ch2)

- 합 규칙: $O(f) + O(g) = O(\max(f, g))$
- 곱 규칙: $O(f) \cdot O(g) = O(f \cdot g)$
- 로그 항등식: $\log_a n = \Theta(\log_b n) \rightarrow \text{점근 표기에서 로그 밑은 무의미}$
- 상수배 무시: $O(cf) = O(f)$ ($c > 0$ 상수)

2-6. 재귀 점화식 감각 (반드시 암기)

🔗 점화식이 뭔가요?

$T(n)$ = "입력 크기 n 일 때 걸리는 시간"

점화식 = "큰 문제 $T(n)$ 을 더 작은 문제 몇 개 + 추가 작업으로 표현"

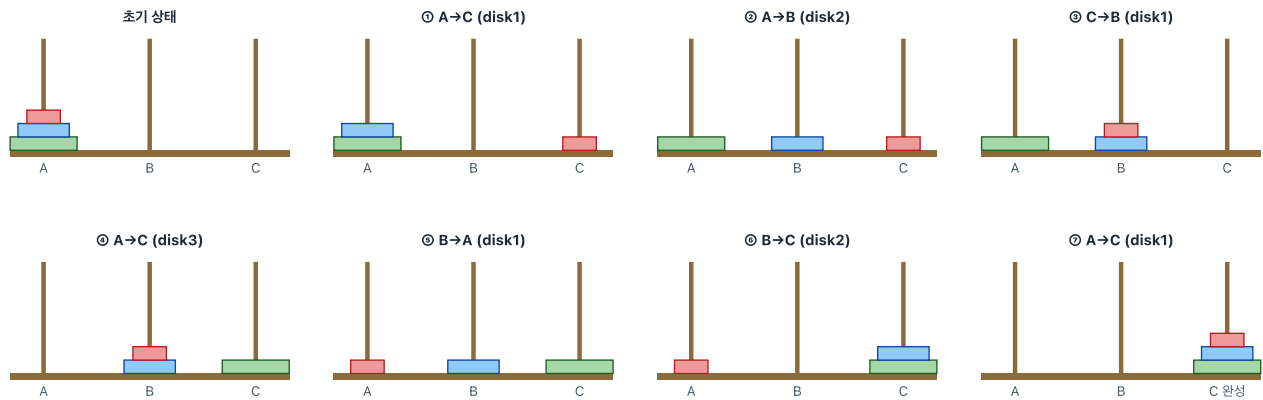
예) $T(n) = T(n/2) + 1$ 읽는 법:

"절반짜리 문제 1개 풀고, 추가로 비교 1회만 더 하면 끝" $\rightarrow$ 매 단계 반씩 줄이니까 로그!

점화식	복잡도	대표 예	일상 비유
$T(n) = T(n-1) + 1$	$O(n)$	선형 재귀	100층 계단을 한 칸씩
$T(n) = T(n/2) + 1$	$O(\log n)$	이진 탐색	100쪽 책에서 반씩 접어가며 찾기
$T(n) = 2T(n/2) + n$	$O(n \log n)$	병합 정렬	반씩 나눠 풀고 다시 합치는 비용 n
$T(n) = 2T(n-1) + 1$	$O(2^n)$	하노이 탑	한 단계마다 경우의 수가 2배 폭발

☞ 하노이 탑 ($n=3$): 7단계의 상태 전이 + 점화식 $T(n) = 2T(n-1) + 1$

하노이 탑 ($n=3$, A→C): 7단계의 상태 전이



점화식: $T(n) = 2 \cdot T(n-1) + 1$, $T(1) = 1$

아이디어 3단계: (1) 위 $n-1$ 개를 보조 기둥으로 옮긴다 (2) 큰 원반을 목표로 (3) $n-1$ 개를 다시 목표로

전개: $T(n) = 2(2T(n-2)+1) + 1 = 4T(n-2) + 3 = \dots = 2^{n-1} \cdot T(1) + (2^{n-1} - 1)$

닫힌해: $T(n) = 2^n - 1$ · $n=3$ 일 때 $T(3) = 7$ ✓ (위 7단계와 일치)

복잡도: $O(2^n)$ — 지수 시간. 원반 64개면 약 1.8×10^{19} 회.

전략 분류: 축소정복 변형(한 원반을 떼어내지만 $2T(n-1)$ 이므로 분할정복 경계선)

3. 억지기법과 완전탐색 (Ch3)

☞ 3장의 본질 — "일단 다 해보기"

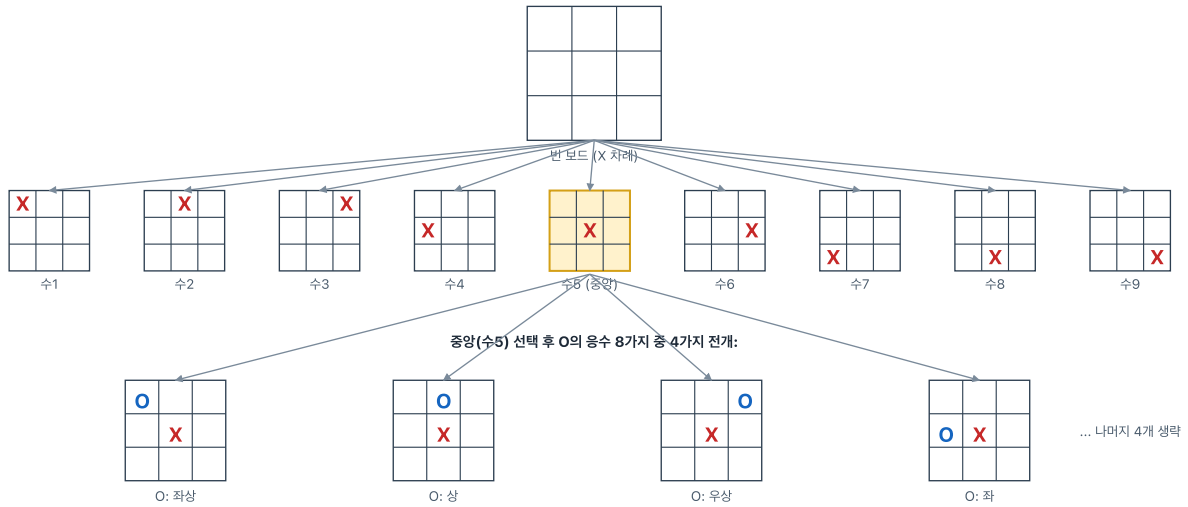
가능한 경우를 하나도 빠짐없이 다 시도하는 가장 정직한 방법

- 비유: 번호 자물쇠 0000~9999를 1번부터 끝까지 돌려보기 — 반드시 맞지만 오래 걸림
- 장점: ✅ 구현 쉽다 ✅ 정답 보장 ✅ 더 똑똑한 방법의 성능 비교 기준선
- 단점: ❌ 입력이 커지면 지수·팩토리얼로 폭발

☞ 상태공간 트리 — 3×3 틱택토로 감 잡기

완전탐색의 핵심 자료구조. 현재 상태 → 가능한 모든 다음 수 → 그 각각의 또 다음 수 ...

3x3 틱택토 상태공간 트리 — 첫수 9가지 → 깊이 2까지 일부 전개



상태공간 크기 분석

깊이 1: 9가지 · 깊이 2: $9 \times 8 = 72$ 가지 · 최대 깊이 9 (9칸 모두 채움)
 단순 곱셈 상한: $9! = 362,880$ (실제로는 승리 종료·대칭 제거로 훨씬 적음)
 완전탐색(brute-force)은 모든 가지를 끝까지 전개 → 승/패/무 판정 후 최선수 선택 (미니맥스 원형)
 대칭 8가지(회전4 × 반사2) 제거하면 첫수는 사실상 3가지: 코너/모서리/중앙
 핵심: 상태공간 트리 = "앞으로 가능한 모든 미래"의 트리, 3장 완전탐색의 전형적 구조

3-1. 선택 정렬

선택 정렬: 매 회차마다 "제일 작은 값"을 맨 앞으로 보내기

초기	<table><tr><td>29</td><td>10</td><td>14</td><td>37</td><td>13</td></tr></table>	29	10	14	37	13	← 전체에서 min=10 찾기 교환(29↔10)
29	10	14	37	13			
1회	<table><tr><td>10</td><td>29</td><td>14</td><td>37</td><td>13</td></tr></table>	10	29	14	37	13	← 나머지에서 min=13 교환(29↔13)
10	29	14	37	13			
2회	<table><tr><td>10</td><td>13</td><td>14</td><td>37</td><td>29</td></tr></table>	10	13	14	37	29	← 나머지 min=14 (이미 자리)
10	13	14	37	29			
3회	<table><tr><td>10</td><td>13</td><td>14</td><td>37</td><td>29</td></tr></table>	10	13	14	37	29	← 나머지 min=29 교환(37↔29)
10	13	14	37	29			
완료	<table><tr><td>10</td><td>13</td><td>14</td><td>29</td><td>37</td></tr></table>	10	13	14	29	37	정렬 완료
10	13	14	29	37			

포인트: 비교 횟수는 $n(n-1)/2 \rightarrow$ 항상 $\Theta(n^2)$. 교환은 최대 $n-1$ 회로 적음.
 "최솟값 찾기"를 n 번 반복하는 단순·무식한(억지) 기법의 대표 예.

- 매 단계마다 남은 부분의 최솟값을 맨 앞으로 교환
- 비유: 책장에서 키가 가장 작은 책을 찾아 맨 왼쪽 $\rightarrow$ 반복

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} \Rightarrow \Theta(n^2)$$

입력이 이미 정렬되어 있어도 똑같이 $\Theta(n^2)$

3-2. 순차 탐색

Ch.3 예제 — 순차 탐색 (Sequential Search): 배열 [7, 3, 9, 4, 8, 2, 6] 에서 key 찾기

왼쪽부터 하나씩 비교. 정렬 불필요. 최선 $O(1)$, 최악 $O(n)$, 평균 $O(n)$

1 성공 사례 — key = 4 (배열 4번째 = 인덱스 3에 존재)

7	3	9	4	8	2	6
0	1	2	3 ★	4	5	6

비교 1: $7 \neq 4$
비교 2: $3 \neq 4$
비교 3: $9 \neq 4$

비교 4: $4 = 4 \checkmark$
→ return 3 (인덱스)

2 실패 사례 — key = 5 (배열에 없음) → 최악 n번 비교

7	3	9	4	8	2	6
---	---	---	---	---	---	---

7회 비교 모두 실패
→ return -1 (없음)
최악: $C_{\text{worst}}(n) = n$

3 평균 비교 횟수 분석 (성공 가정, 키가 균등분포로 존재)

$P(\text{key at index } i) = 1/n$ (균등)

비교 횟수(index i에 있을 때) = $i + 1$ (1부터 시작)

$$C_{\text{avg}}(n) = (1/n) \cdot \sum_{i=0}^{n-1} (i+1) = (1/n) \cdot n(n+1)/2 = (n+1)/2$$

성공 확률 p 를 도입하면: $C_{\text{avg}} = p \cdot (n+1)/2 + (1-p) \cdot n$

$p=1$: $(n+1)/2 \approx n/2$ · $p=0$: n → 어느 경우든 $\Theta(n)$

4 의사코드와 개선: 보초(Sentinel) 기법

표준 버전

```
seq_search(A, n, key):
for i = 0 to n - 1:
if A[i] = key: return i
return -1
```

루프마다 2회 비교 ($i < n$, $A[i] = \text{key}$)

보초 버전 (Sentinel)

```
A[n] ← key # 끝에 임시 삽입
i = 0
while A[i] ≠ key: i ← i + 1
return i if i < n else -1
```

경계검사 제거 — 실행시간 약 2배 빠름

앞에서부터 하나씩 비교 → 최선 1, 평균·최악 $O(n)$

3-3. 브루트포스 문자열 매칭

$$T(n, m) = O((n - m + 1)m)$$

느린 이유: 패턴 앞부분이 일치하다 끝에서 실패 → 거의 처음부터 다시 비교

→ 6장 Horspool이 이것을 개선

3-4. 0/1 배낭 (완전탐색)

0/1 배낭 완전탐색: 물건 A(10,60), B(20,100), C(30,120), 용량 50

① 각 물건을 "넣거나 / 안 넣거나" → $2^3 = 8$ 가지 경우를 전부 확인한다.

선택	무게	가치	용량 50 이내?	판정
∅	0	0	O	가장 낮음
A	10	60	O	단일 선택 중 최저
B	20	100	O	단일 선택 중 중간
C	30	120	O	단일 선택 중 최고
A+B	30	160	O	두 개 조합
A+C	40	180	O	두 개 조합
B+C	50	220	O (딱 50)	★ 최대 가치 — 정답
A+B+C	60	280	X (초과)	가치가 높아도 용량 초과 → 탈락

② 정답: B + C, 총 가치 220. (용량을 딱 맞게 채우면서 가치 최대)

③ 왜 "완전탐색"인가: 각 물건마다 "넣/안넣" 2가지 → 경우의 수 2^n . n 이 커지면 폭발.

④ 개선 방향: 7장 동적 계획법(DP)으로 중복 계산을 제거하면 $O(nW)$ 로 줄어든다.

물건 $A = (10, 60)$, $B = (20, 100)$, $C = (30, 120)$, 한계 50

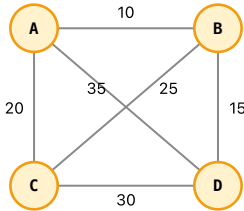
- 정답: $B + C$, 가치 220
- 경우의 수: 2^n — 물건이 많아지면 불가능
- → 7장 DP로 개선

3-5. TSP (외판원 문제)

Ch.3 예제 — 외판원 문제(TSP) 완전탐색 (4개 도시)

도시 A를 출발해 B,C,D를 모두 한 번씩 방문 후 A로 복귀 — 최소 비용 경로를 모든 순열 검사로 찾기

1 가장 그래프와 비용 행렬



비용 행렬 (대칭):

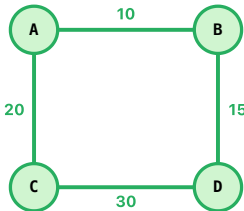
	A	B	C	D
A	0	10	20	25
B	10	0	35	15
C	20	35	0	30
D	25	15	30	0

고정: 출발 A
나머지 {B, C, D}의 순열 = $3! = 6$
각 순열 뒤에 A 붙여 완전 경로 구성
순열:
B-C-D, B-D-C, C-B-D,
C-D-B, D-B-C, D-C-B
대칭 경로 중복 → 실질 3가지

2 모든 경로 비용 계산 — 각 순열에 대해 4개 간선 합산

#	경로	비용 계산	합계
1	A → B → C → D → A	10 + 35 + 30 + 25	100
2	A → B → D → C → A	10 + 15 + 30 + 20	75 * 최솟값
3	A → C → B → D → A	20 + 35 + 15 + 25	95
4	A → C → D → B → A	20 + 30 + 15 + 10	75 * (#2의 역방향)
5	A → D → B → C → A	25 + 15 + 35 + 20	95 (#3의 역방향)
6	A → D → C → B → A	25 + 30 + 35 + 10	100 (#1의 역방향)

3 최적 경로 시각화 — A → B → D → C → A (비용 75)



복잡도 분석 — 왜 느린가
출발지 고정 → $(n-1)!$ 순열 · 각 순열 비용 합산 $\Theta(n)$
⇒ $T(n) = \Theta((n-1)! \cdot n) = \Theta(n!)$
 $n=10 \rightarrow$ 약 360만 경로 · $n=15 \rightarrow$ 약 870억 · $n=20 \rightarrow 10^{18}$
개선: 동적계획법 (비트마스크 DP) $\Theta(n^2 \cdot 2^n)$ — 20-도시급까지 가능
실전: 근사 알고리즘 (2-OPT, 최근접 이웃) — 실용적 해

출발 도시 고정 시 후보 순열 수:

$(n - 1)!$

↻ 대칭 TSP면 왕복 중복 제거로 $\frac{(n - 1)!}{2}$

$n = 10$ 이면 $9! = 362,880 \rightarrow$ 대규모 TSP 완전탐색 불가

3-5-α. 작업 배정 Job Assignment (완전탐색)

n 명× n 작업, 각 사람이 서로 다른 작업을 맡는 순열 중 총 비용 최소를 찾는 문제. 경우의 수 $n!$ — TSP와 동일한 팩토리얼 폭발.

작업 배정 (Job Assignment): 4명×4작업 비용 행렬, 4! = 24 순열 중 3가지 시도

비용 행렬 C[사람][작업]				
	작업1	작업2	작업3	작업4
사람A	9	2	7	8
사람B	6	4	3	7
사람C	5	8	1	8
사람D	7	6	9	4

문제 & 완전탐색 전략

각 사람은 정확히 한 작업, 각 작업은 정확히 한 사람에게 배정.
 배정 = 작업 번호의 순열 (예: A→1, B→2, C→3, D→4)
 경우의 수 = $n! = 4! = 24$ 가지 (모두 시도)
 총 비용 = $C[A][j1] + C[B][j2] + C[C][j3] + C[D][j4]$
 시간 복잡도 = $\Theta(n \cdot n!)$ - 각 순열당 n개 비용 합산

시도 ① 순열 (1,2,3,4): A→1, B→2, C→3, D→4

9 + 4 + 1 + 4 = 18
 해당 순열

시도 ② 순열 (2,1,3,4): A→2, B→1, C→3, D→4

2 + 6 + 1 + 4 = 13
 현재 최선 후보

시도 ③ 순열 (4,3,2,1): A→4, B→3, C→2, D→1

8 + 3 + 8 + 7 = 26
 역순 순열

... 24개 순열 중 전체 최소는 다음 순열:

순열 (2,3,1,4): A→2, B→3, C→1, D→4
 비용 2 + 3 + 5 + 4 = 14 ← 최소

요약 & 개선 여지

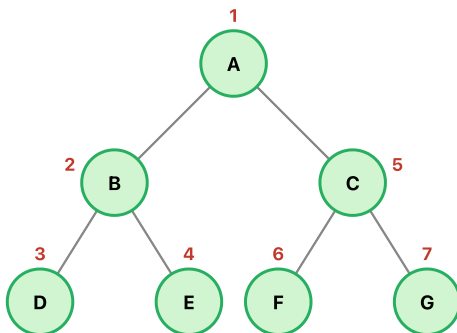
$n=4 \rightarrow 24$ 가지 / $n=10 \rightarrow 3,628,800$ 가지 / $n=20 \rightarrow 2.4 \times 10^{18}$ 가지 - 완전탐색 불가
 → 8장 탐욕(근사), 9장 분기한정(하한으로 가지치기)로 개선 가능
 최적해 보장 대신 시간이 기하급수적 - 완전탐색의 전형적 딜레마

- 완전탐색 시간: $\Theta(n \cdot n!)$
- 8장 탐욕(근사해)·9장 분기한정(하한으로 가지치기)로 개선

3-6. 그래프 DFS / BFS

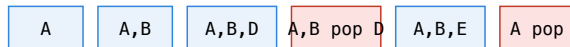
교재 예(A~H 8-node)로 보는 탐색 흐름

DFS (깊이 우선) — 스택 / 재귀



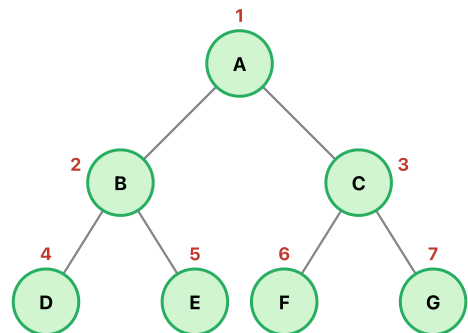
방문 순서: A → B → D → E → C → F → G
 "한 방향으로 끝까지, 막히면 백트랙"

스택 변화 (push=내려가기 / pop=백트랙)



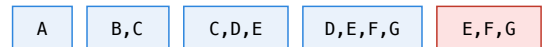
계속 이어짐: [A,C] → [A,C,F] → [A,C,G] → ...

BFS (너비 우선) — 큐



방문 순서: A → B → C → D → E → F → G
 "레벨 0 → 레벨 1 → 레벨 2 (동시 전진)"
 거리: A=0, B/C=1, D/E/F/G=2

큐 변화 (enqueue 뒤 / dequeue 앞)



앞에서 꺼내 확장 → 뒤에 추가. "최단 거리" 보장

DFS vs BFS 요약

항목	DFS (깊이 우선)	BFS (너비 우선)
자료구조	스택 (또는 재귀 호출)	큐 (FIFO)
탐색 순서	한 경로 끝까지 → 백트랙	레벨 단위로 동시 확장
주요 용도	사이클·연결성·위상정렬·백트래킹	가중치 없는 최단경로, 레벨 기반
공간	$O(V)$ — 재귀 깊이만큼	$O(V)$ — 한 레벨 크기
공통:	시간 $O(V+E)$, 방문 체크 필수. 인접리스트 권장.	

그래프 DFS / BFS — 시작 정점 ⑧, 이웃은 번호 작은 것부터

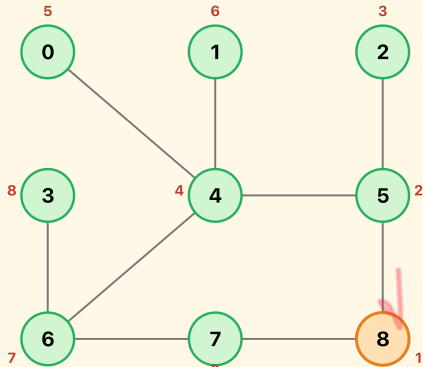
인접 리스트

0: {4} 1: {4} 2: {5} 3: {6} 4: {0,1,5,6}
5: {2,4,8} 6: {3,4,7} 7: {6,8} 8: {5,7} ← 시작

※ 정점 9개, 간선 9개 (0-4, 1-4, 2-5, 3-6, 4-5, 4-6, 5-8, 6-7, 7-8)
※ PDF 3장 26p 필기 예제

DFS (깊이 우선) — 재귀 / 스택

방문 순서 번호는 노드 위에 빨간색으로 표시



방문 순서

8 → 5 → 2 → 4 → 0 → 1 → 6 → 3 → 7

※ 각 정점에서 이웃 중 번호 작은 것부터 재귀 호출 → 막히면 되돌아감

DFS 재귀 호출 스택 변화 (왼쪽 bottom → 오른쪽 top)

현재 visit 노드는 주황색, 방문 끝나 pop 되면 사라짐

① visit 8

② 8→5

③ 5→2

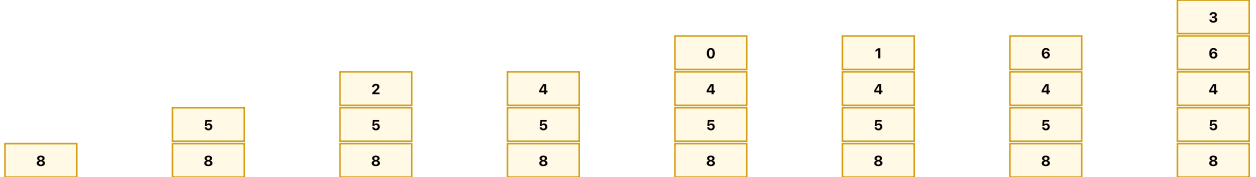
④ 5→4

⑤ 4→0

⑥ 4→1

⑦ 4→6

⑧ 6→3

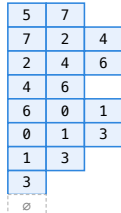


⑧ 6→7 (3 pop 후): 스택 [8,5,4,6,7] → 더 갈 곳 없어 전체 pop → 종료

BFS 큐 변화 (왼쪽 front → 오른쪽 rear)

각 단계에서 front를 dequeue → visit → 이웃 중 미방문 정점을 번호순으로 enqueue

- ① dequeue 8, enqueue 5,7
- ② dequeue 5, enqueue 2,4
- ③ dequeue 7, enqueue 6
- ④ dequeue 2
- ⑤ dequeue 4, enqueue 0,1
- ⑥ dequeue 6, enqueue 3
- ⑦ dequeue 0
- ⑧ dequeue 1
- ⑨ dequeue 3 (종료)



최종 방문 순서



※ 큐가 빌 때까지 반복. 인접 리스트 시 $O(V+E)$.

알고리즘	자료구조	특징	복잡도(인접 리스트)
DFS	스택 / 재귀	미로처럼 깊게, 사이클 검사·위상정렬 유리	$O(V + E)$
BFS	큐	레벨 순서, 가중치 없는 최단 경로	$O(V + E)$

인접 행렬일 땐 둘 다 $O(V^2)$

손계산 팁 (시험 대비)

- 이웃은 번호(알파벳) 작은 것부터 방문 — 교수님 강의 규칙
- DFS**: 현재 정점에서 미방문 이웃 중 가장 작은 것으로 계속 내려감 → 막히면 backtrack
- BFS**: 현재 정점의 미방문 이웃을 번호 작은 것부터 큐에 enqueue → 큐의 front부터 dequeue
- 위 9-node 예: 시작 ⑧ → DFS 순서 8,5,2,4,0,1,6,3,7 / BFS 순서 8,5,7,2,4,6,0,1,3

3-7. 해밀토니안 회로 vs 오일러 회로 (PDF Ch3)

빈출 구분 문제 ★

항목	해밀토니안 회로	오일러 회로
조건	모든 정점을 정확히 한 번씩 지나 출발점 복귀	모든 간선을 정확히 한 번씩 지나 출발점 복귀
판정	NP-완전 (일반해 없음)	모든 정점의 차수가 짝수이면 존재 (오일러 정리)
대표 문제	TSP(외판원)	코니히스베르크 다리
완전탐색 복잡도	$O(n!)$	$O(V + E)$ 판정 가능

4. 축소정복 Decrease-and-Conquer (Ch4)

🔗 4장의 본질 — "문제를 한 번에 한 조각씩 깎아낸다"

큰 문제를 더 작은 문제 "하나"로 줄여서 반복

- 비유: 사과 껍질을 한 바퀴 돌려가며 벗기기 — 매번 조금씩 작아지다가 사라짐
- 분할정복과의 차이: 분할정복은 "두 개로 쪼개기", 축소정복은 "하나 떼어내기"

4-1. 축소 방식 3가지

방식	예
고정 크기 ($n \rightarrow n - 1$)	팩토리얼, 삽입 정렬
고정 비율 ($n \rightarrow n/2$)	이진 탐색, 빠른 거듭제곱
가변 크기	Quick Select, 유클리드

4-2. 삽입 정렬

삽입 정렬: 카드 한 장씩 "이미 정렬된 왼쪽"으로 끼워 넣기

초기

5	2	4	6	1	3
---	---	---	---	---	---

→ 왼쪽 한 장(5)은 이미 정렬

2 삽입

2	5	4	6	1	3
---	---	---	---	---	---

→ 2는 5 앞으로 이동

4 삽입

2	4	5	6	1	3
---	---	---	---	---	---

→ 4는 2와 5 사이로 끼움

6 삽입

2	4	5	6	1	3
---	---	---	---	---	---

→ 6은 이미 제자리

1 삽입

1	2	4	5	6	3
---	---	---	---	---	---

→ 1은 맨 앞으로

포인트: 이미 정렬된 경우 비교만 하고 끝나 최선 $O(n)$. 최악(역정렬)은 $O(n^2)$.
 "작은 문제($n-1$)까지 해결 + 한 장만 추가"로 푸는 축소정복의 대표 예.

왼쪽의 이미 정렬된 부분에 새 원소를 끼워 넣음

손계산: [5, 2, 4, 6, 1, 3]

단계	정렬 구간	삽입	결과
1	[5]	2	[2, 5, 4, 6, 1, 3]
2	[2, 5]	4	[2, 4, 5, 6, 1, 3]
3	[2, 4, 5]	6	변화 없음
4	[2, 4, 5, 6]	1	[1, 2, 4, 5, 6, 3]
5	[1, 2, 4, 5, 6]	3	[1, 2, 3, 4, 5, 6]

- 최선(이미 정렬): $O(n)$ — 비교만, 이동 없음 ★
- 최악(역순): $O(n^2)$

거의 정렬된 입력에 특히 강함

🔄 변형: 이진 삽입 정렬 — 위치 탐색만 이분탐색으로

이미 정렬된 왼쪽에 새 원소를 넣을 때 순차 비교 대신 이진 탐색을 써서 $O(\log k)$ 에 삽입 위치 결정. 단, 이동은 여전히 $O(k)$ 이므로 전체는 $\Theta(n^2)$.

이진 삽입 정렬: 정렬된 왼쪽에 새 원소를 이분탐색으로 위치 결정 → 이동

㉠ 현재 상태: 정렬된 왼쪽 [2, 4, 5, 7, 9] + 삽입할 원소 6



㉡ 이분탐색 Step 1: $l=0, r=4, mid=2 \rightarrow A[2]=5 < 6 \rightarrow$ 오른쪽 [3..4]으로



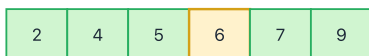
㉢ 이분탐색 Step 2: $l=3, r=4, mid=3 \rightarrow A[3]=7 > 6 \rightarrow$ 왼쪽 [3..2]로 (종료)



㉣ 삽입 위치 = 인덱스 3 (5와 7 사이)



㉤ 삽입 완료



복잡도 비교

위치 찾기: 일반 삽입정렬 $O(k)$ → 이진삽입 $O(\log k)$ · 이동: 여전히 $O(k)$ (원소를 한 칸씩 밀어야 함)
총 비교: $\Theta(n \log n)$ · 총 이동: $\Theta(n^2) \Rightarrow$ 비교 비용이 비쌀 때만 유리 (전체는 여전히 $\Theta(n^2)$)

4-3. 위상 정렬 — DFS 기반 (알파벳순 방문)

🔄 전제 조건: DAG(사이클 없는 방향 그래프)

사이클 있으면 "누가 먼저"인지 모순

⚠️ 시험 풀이 규칙 (교수님 지정)

- 인접 정점은 알파벳 순으로 방문
- DFS 기반 방법만 사용 (진입차수 방식 X)
- 자식이 더 없는 정점부터 **finish** 처리 → 스택에 **push**
- 마지막에 스택을 역순(**reverse**)으로 **pop** → 위상 정렬 결과

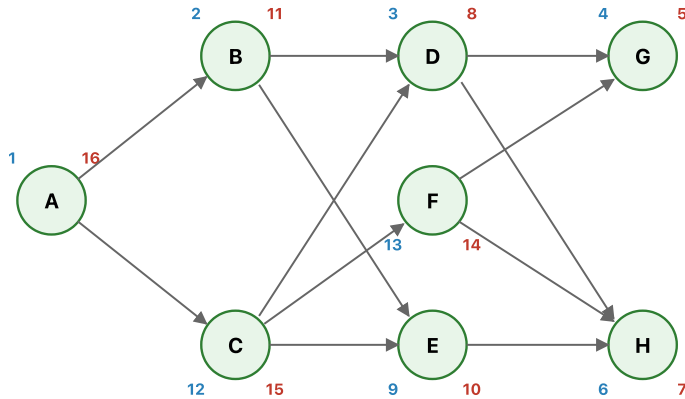
알고리즘 3단계

- 1. DFS를 호출하며 각 정점의 방문시각 $d[v]$ / 종료시각 $f[v]$ 기록
- 2. 정점의 모든 인접 정점을 다 본 순간(더 내려갈 곳이 없을 때) **finish** → 스택에 **push**
- 3. 모든 DFS가 끝나면 스택을 **pop**(역순) 한 순서가 정답

DFS 기반 위상 정렬 — 알파벳순 방문, finish 역순 출력

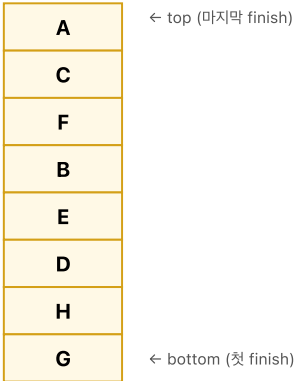
간선: $A \rightarrow B, C$ $B \rightarrow D, E$ $C \rightarrow D, E, F$ $D \rightarrow G, H$ $E \rightarrow H$ $F \rightarrow G, H$

㉠ 그래프 + 방문/종료 시각 (d / f)



㉡ Finish 스택

(종료 순서대로 push)



↓ reverse (pop)

㉢ DFS 탐색 순서 (알파벳 우선)

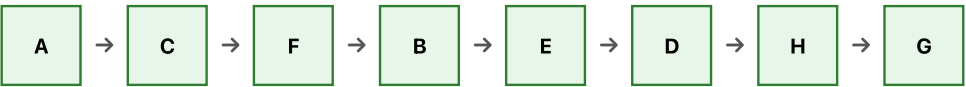
$A \rightarrow B \rightarrow D \rightarrow G[f=5] \rightarrow H[f=7] \rightarrow D[f=8]$

$\rightarrow E[f=10] \rightarrow B[f=11] \rightarrow C \rightarrow F[f=14] \rightarrow C[f=15] \rightarrow A[f=16]$

※ 각 정점에서 인접 정점이 여러 개면 알파벳 작은 것부터 방문

※ 더 갈 곳이 없으면 **finish**하고 스택에 push

㉣ 스택 pop (역순) = 위상 정렬 결과



f 값이 큰 순서대로 정렬한 것과 동일 (16, 15, 14, 11, 10, 8, 7, 5)

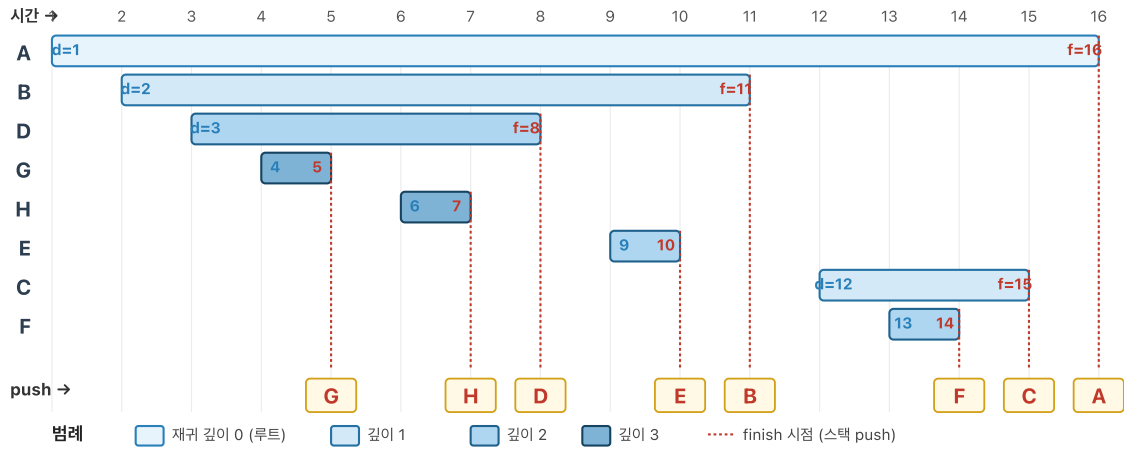
예제 그래프 (PDF Ch4, 15p)

$A \rightarrow B, C \mid B \rightarrow D, E \mid C \rightarrow D, E, F \mid D \rightarrow G, H \mid E \rightarrow H \mid F \rightarrow G, H$

DFS 타임라인 — 방문(d)/종료(f) 구간의 중첩 구조

DFS 타임라인 — 방문(d) / 종료(f) 구간의 중첩 구조

바깥 구간이 자식 구간을 포함(nesting) — 재귀 깊이 = 현재 열려있는 바의 개수



① Finish 스택 (push 순서, 원→오른쪽이 아래→위)



reverse ↓

↑ bottom (먼저 finish) top (마지막 finish) ↑

② 역순 = 위상 정렬 결과



f 값이 큰 순 (16 → 15 → 14 → 11 → 10 → 8 → 7 → 5)

- 각 정점의 바 = [d, f] 구간 → 바깥이 자식을 포함하면 부모-자식 관계
- 빨간 점선 = finish 시점 → 스택에 push
- 스택을 reverse 한 결과가 위상 정렬: A C F B E D H G

🔗 시험 팁 — 암기 패턴

1. 그래프 옆에 d/f 두 칸을 적어두기
2. 방문할 때마다 d 증가, finish할 때마다 f 증가 (같은 카운터 공유)
3. f 번호가 큰 순서대로 나열하면 그게 바로 위상 정렬

🔗 함정

- 위상 정렬 결과는 유일하지 않음 — 인접 정점을 다른 순서로 방문하면 다른 답도 가능하지만, 알파벳 순 규칙 하에서는 답이 하나로 결정됨
- 시간복잡도: $O(V + E)$

4-4. 이진 탐색

이진 탐색: 정렬된 9개 배열에서 target = 23 찾기

매 반복마다 $mid = (low+high)/2$ → 한쪽 절반을 버림 → 후보 $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$

1 low=0, high=8, mid=4

0	1	2	3	4	5	6	7	8
3	7	11	17	20	23	29	35	41

low mid=20 high

20 < 23 → 오른쪽 반 탐색
low = mid+1 = 5

판단 규칙

A[mid] == target → 찾음 ✓
A[mid] < target → low = mid+1
A[mid] > target → high = mid-1
종료 조건: low > high (없음)

한 번 비교 = 후보 절반 삭제

2 low=5, high=8, mid=6

3	7	11	17	20	23	29	35	41
---	---	----	----	----	----	----	----	----

low mid=29 high

29 > 23 → 왼쪽 반 탐색
high = mid-1 = 5

3 low=5, high=5, mid=5 → A[5]=23 ✓ 발견!

3	7	11	17	20	23	29	35	41
---	---	----	----	----	----	----	----	----

return 5 (인덱스)

low=mid=high=5 → target 일치 ✓

복잡도 $T(n)=T(n/2)+O(1) \Rightarrow O(\log n)$

- 전제조건: (1) 배열이 정렬됨 (2) 인덱스 $O(1)$ 랜덤 접근 (링크드리스트는 불가)
- 직관: 스무고개처럼 질문 한 번에 후보가 반씩. $n=1,000,000 \rightarrow$ 약 20번이면 끝.
- 오버플로 주의: $mid = low + (high-low)/2$ (단순 $(low+high)/2$ 은 큰 n 에서 오버플로 위험)

중앙 원소와 비교해 절반을 버림

$$T(n) = T(n/2) + 1 \Rightarrow O(\log n)$$

전제: 리스트가 정렬되어 있어야 함

비유 — 스무고개: 1~100 중 "50보다 큰가요?"로 절반씩 걸러내기

손계산: [3, 8, 12, 17, 24, 31, 37, 42, 55] 에서 37 찾기

단계	구간	중앙	판단
1	전체	24	37>24, 오른쪽
2	[31, 37, 42, 55]	37	찾음

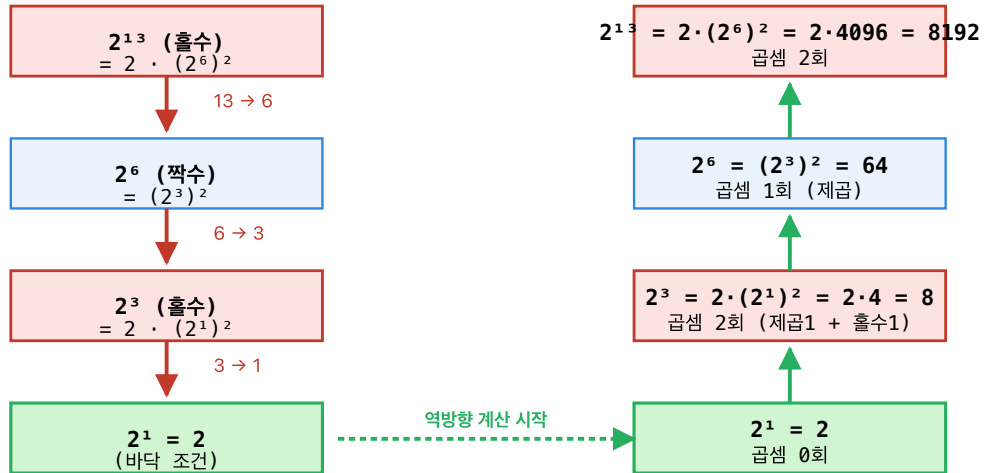
4-5. 빠른 거듭제곱

빠른 거듭제곱 (Exponentiation by Squaring): 2^{13} 계산

규칙: 짝수 $n \rightarrow a^n = (a^{n/2})^2$ | 홀수 $n \rightarrow a^n = a \cdot (a^{(n-1)/2})^2$ | 한 단계마다 지수가 반으로 $\Rightarrow O(\log n)$

↓ 분해 (지수를 절반으로)

↑ 재조합 (값 계산)



곱셈 횟수 비교

- 무식한 방법 (2를 12번 곱함): 곱셈 12회
- 빠른 거듭제곱: $0 + 2 + 1 + 2 = 5$ 회 $\Rightarrow O(\log_2 13) \approx 4$ 수준
- 이진수 시각화: $13 = 1101_2 \rightarrow$ 비트가 1이면 "곱셈 한 번 더" 추가됨 (홀수 처리와 동일)
- 활용: RSA 모듈러 거듭제곱 $a^n \bmod m$ — 큰 n (1024-bit)에서 필수

이진수 관점: $13 = 1101_2$ — 왼쪽 $\rightarrow$ 오른쪽 읽기

비트 1: result = 1 $\rightarrow$ result² $\cdot$ 2 = 2 (첫 비트)
 비트 1: result = 2 $\rightarrow$ 2² $\cdot$ 2 = 8 (홀수)
 비트 0: result = 8 $\rightarrow$ 8² = 64 (짝수 $\rightarrow$ 제곱만)
 비트 1: result = 64 $\rightarrow$ 64² $\cdot$ 2 = 8192 $\checkmark$ 정답

- n 짝수: $a^n = (a^{n/2})^2$
- n 홀수: $a^n = a \cdot (a^{(n-1)/2})^2$

$$T(n) = T(n/2) + O(1) \Rightarrow O(\log n)$$

예: $2^{13} = 2 \cdot (2^6)^2$, $2^6 = (2^3)^2$, $2^3 = 2 \cdot (2^1)^2 \rightarrow$ 곱셈 4~5회로 끝

4-6. Quick Select — k 번째 작은 수

Quick Select — k번째로 작은 수 찾기 (부분 정렬로 충분)

배열 [7, 4, 6, 3, 9, 1] 에서 k = 3번째로 작은 수를 찾자 (미리보기: 정렬 시 [1, 3, 4, 6, 7, 9] → 답 = 4)

핵심 규칙 — 피벗 분할 후 "피벗의 순위 r"과 "찾을 순위 k"를 비교

분할이 끝나면 피벗은 정렬 시 자기 자리에 놓임 → 피벗 앞에 있는 원소 수 = r-1 → 이로부터 **피벗 순위 r** 을 즉시 알 수 있음

① $r = k$ → 피벗이 답! 종료
(피벗 자리에 k번째가 딱 놓인 경우)

② $r > k$ → 답은 피벗 왼쪽에 존재
왼쪽 부분에서 동일한 k 그대로 탐색

③ $r < k$ → 답은 피벗 오른쪽에 존재
오른쪽에서 $(k - r)$ 번째 를 탐색

0 초기 배열 (길이 n = 6, 찾을 순위 k = 3)

[0]	[1]	[2]	[3]	[4]	[5]
7	4	6	3	9	1

↑ 피벗 = 7 (첫 원소 선택)

- 목표: 3번째로 작은 수
- 전략: 분할 후 피벗 순위 r 을 확인 → 규칙 적용 → 한쪽만 재귀
- 반드시 전체 정렬할 필요 없음 ← 여기서 log 인수가 사라지는 핵심

1 피벗 7로 분할 → 7 보다 작은 값 5개가 앞으로, 9가 뒤로 이동 → 7은 인덱스 [5] 자리 확정

[0]	[1]	[2]	[3]	[4]	[5]
4	3	1	6	9	7

← 4개 모두 < 7 (왼쪽 그룹)

9 > 7 pivot 자리 확정

판단

피벗 7은 인덱스 [5] → 앞에 5개 존재 → **순위 r = 6**
 $r = 6, k = 3 \rightarrow r > k$ → 규칙 ② 적용
 → **왼쪽 부분 [4, 3, 1, 6]** 에서 계속 k=3 그대로 탐색 (9·7 버림)

2 남은 [4, 3, 1, 6] 에서 피벗 4로 분할 → 4는 인덱스 [2] 자리 확정

[0]	[1]	[2]	[3]	[4]	[5]
3	1	4	6	9	7

< 4 (2개)

pivot r=3 > 4 (1개) (1단계에서 이미 탈락)

판단 — 정답 발견!

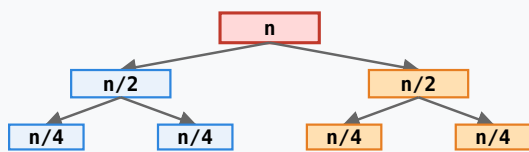
피벗 4는 인덱스 [2] → 앞에 2개 존재 → **순위 r = 3**
 $r = 3, k = 3 \rightarrow r = k$ → 규칙 ① 적용
 → **피벗 4 가 바로 답** — 재귀 종료 (왼쪽·오른쪽 들어갈 필요 없음)

✓ 결과: [7, 4, 6, 3, 9, 1] 의 3번째로 작은 수 = 4

전체 정렬 [1, 3, 4, 6, 7, 9] 중 3번째 = 4 ✓. 단 2번의 분할만으로 정답 도달 (오른쪽 절반은 한 번도 건드리지 않음)

왜 평균 복잡도가 $O(n)$ 인가? — 퀵정렬과 재귀 구조 비교

퀵정렬: 양쪽 다 재귀 → $T(n) = 2T(n/2) + n$



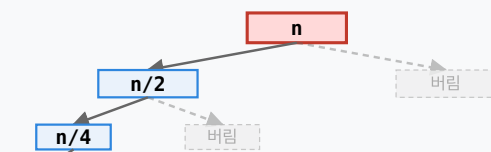
... 각 레벨 작업량 합 = n

레벨 수 $\log n$ → 전체 $n \cdot \log n$

양쪽 전부 정렬이 필요해 어느 가지도 버릴 수 없음

△ 최악 $O(n^2)$: 피벗이 매번 극단값(최소/최대)으로 뽑혀서 크기가 n-1 씩만 줄어들 때 → 무작위 피벗 또는 median-of-medians 로 회피

Quick Select: 한쪽만 재귀 → $T(n) = T(n/2) + n$



$n + n/2 + n/4 + n/8 + \dots \leq 2n$ (등비급수)

평균 $O(n)$ — log 인수가 사라짐

1. 피벗으로 분할 → 피벗의 최종 위치 p

2. $p = k$ → 답 / $p > k$ → 왼쪽 재귀 / $p < k$ → 오른쪽 재귀

손계산: [7, 4, 6, 3, 9, 1] 에서 3번째로 작은 수 → 4

경우	복잡도
평균	$O(n)$
최악	$O(n^2)$

4-7. 보간 탐색 Interpolation Search (PDF Ch4)

이진 탐색의 "무조건 중앙"이 아닌, **값의 분포**를 추정해 탐색 위치 결정:

$$x = l + \left\lfloor \frac{(K - A[l])(r - l)}{A[r] - A[l]} \right\rfloor$$

- 균등 분포일 때: 평균 $O(\log \log n)$
- 편중 분포일 때: 최악 $O(n)$
- 비유: 전화번호부에서 ㅇ 은 뒤쪽을 먼저 펼치기

4-8. 위조동전 찾기 (PDF Ch4)

n 개 중 가벼운 위조 1개를 양팔저울로 찾기 — 축소정복 전형:

1. n 을 3등분하여 두 그룹을 저울에 올림
2. 기울어지면 가벼운 쪽, 수평이면 나머지 그룹 $\rightarrow n/3$ 로 축소
3. $T(n) = T(n/3) + 1 \Rightarrow \Theta(\log_3 n)$

⚠ 시험 포인트 — 가짜가 가볍거나 무거울 수 있을 때

교재는 "가짜 = 가벼움" 전제이지만, 방향을 모를 때는 풀이가 달라진다.

핵심 아이디어 (정보이론적 하한)

- 양팔저울 1회 측정의 결과는 **3가지** (좌>우, 좌=우, 좌<우)
 $\rightarrow k$ 회 측정 시 구분 가능한 결과 = 3^k
- 구분해야 할 경우의 수: 각 동전이 "가짜+가벼움" 또는 "가짜+무거움" $\rightarrow 2n$

$$3^k \geq 2n \implies k \geq \lceil \log_3(2n) \rceil$$

즉 최소 측정 횟수 $T(n) = \lceil \log_3(2n) \rceil$

n (동전 수)	$2n$	최소 측정 횟수 k	가벼움 전제였다면
3	6	2번 ($3^2 = 9$)	1번
4	8	2번	2번
5~13	10~26	3번 ($3^3 = 27$)	2~3번
12	24	3번 $\leftarrow$ 고전 12동전 문제	3번
14~40	28~80	4번 ($3^4 = 81$)	3~4번

🔄 비교 암기

- 가벼움 전제: $T(n) = \lceil \log_3 n \rceil$
- 방향 모름: $T(n) = \lceil \log_3(2n) \rceil$ — 대략 1단계씩 증가
- 이유: 구분해야 할 상태가 $n \rightarrow 2n$ 로 두 배가 되기 때문

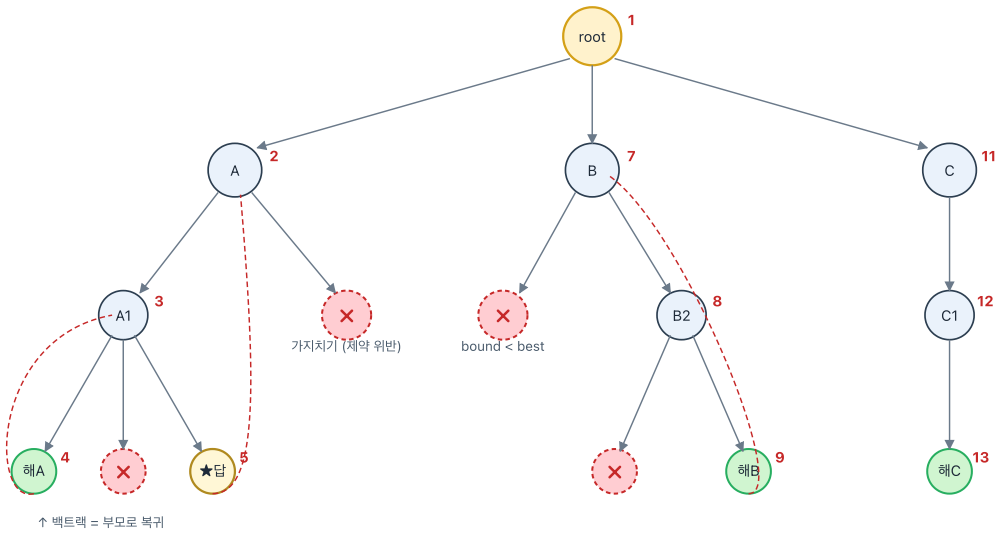
4-9. 유클리드 호제법 (PDF Ch1·Ch4 연계)

$$\gcd(m, n) = \gcd(n, m \bmod n), \quad \gcd(m, 0) = m$$

- 가변 크기 축소정복의 대표 예
- 복잡도: $O(\log(\min(m, n)))$ — 연속 두 단계에서 값이 절반 이하로 줄어듦(Lamé 정리)

4-10. 축소정복 + 백트래킹(선행 개념)

축소정복의 상태공간 트리 탐색은 결국 **DFS + 가지치기** 구조. 본격 다루는 Ch9의 맛보기:



알고리즘 개요

DFS 방문 순서 (빨간 번호): 1(root) → 2(A) → 3(A1) → 4(해A) → 5(★답) → ... → 13(해C)
 × 표시 = 제약 위반 or bound-best (분기한정) → 그 가지 전체 탐색 생략
 점선 빨간 화살표 = 백트랙(부모로 복귀) → 현재 노드 탐색 or 조부모로 다시 복귀
 최악 $O(b^d)$ (분기수^{깊이})이지만 가지치기로 실제 탐색 공간은 크게 축소됨

- DFS 순서로 상태공간 트리 전개, 제약 위반·분기한정(bound) 시 즉시 되돌아감(backtrack)
- 가지치기 덕에 최악 $O(b^d)$ 상한보다 훨씬 적게 탐색하는 경우가 많음

5. 분할정복 Divide-and-Conquer (Ch5)

🔗 5장의 본질 — "나눠서 각개격파 후 합치기"

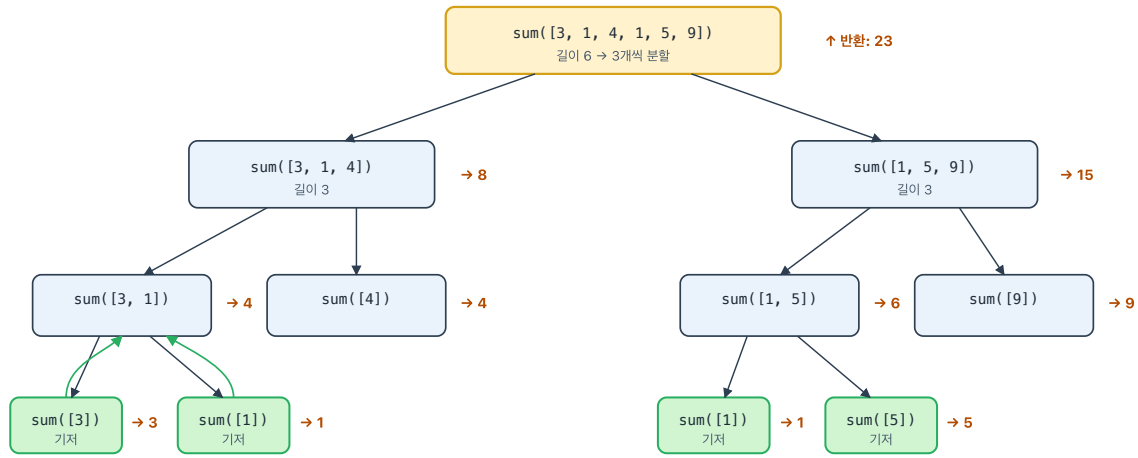
큰 문제를 여러 개의 작은 문제로 나눠 각각 풀고 결과를 합친다

- 비유: 반 학생 40명의 성적표 정리 = 두 팀 20명씩 맡겨 정리시킨 뒤, 둘을 합쳐 최종 정리
- 3단계: ① **Divide**(나누기) → ② **Conquer**(작은 문제 해결) → ③ **Combine**(합치기)
- 축소정복과의 차이: 축소는 한 조각 떼어내기, 분할은 두 개 이상으로 쪼개기

☞ 입문 예: 리스트 합 [3, 1, 4, 1, 5, 9]

좌/우 반으로 나눠 각각 합한 뒤 더함 → $T(n) = 2T(n/2) + O(1) \Rightarrow \Theta(n)$

리스트 합 분할정복: [3,1,4,1,5,9] → 좌/우 분할 → 합쳐 올라오기



의사코드

```

sum(A[lo..hi]):
  if lo == hi:
    return A[lo] ← 기저
  mid = (lo + hi) / 2
  L = sum(A[lo..mid])
  R = sum(A[mid+1..hi])
  return L + R ← 결합
  
```

복잡도 분석 (마스터 정리)

점화식: $T(n) = 2 \cdot T(n/2) + O(1)$
 $a=2, b=2, d=0 \rightarrow b^d = 1, a=2 > 1$ (case 3)
 $\Rightarrow T(n) = \Theta(n^{\log_2 2}) = \Theta(n)$
 잎(leaf) 개수: n 개 = 덧셈 $n-1$ 회 = 반복문과 동일
 최종 합: $3+1+4+1+5+9 = 23$

5-1. 마스터 정리 — 결과만 기억

△ 교수님 공지 ★

마스터 정리의 유도·증명·case 분석은 시험 범위 아님 (어려운 내용이라 제외).
 분할정복 알고리즘의 점근적 복잡도 결과만 기억하면 됨.

$$T(n) = aT(n/b) + f(n)$$

시험에서 쓰는 결과표 — 이것만 외우자

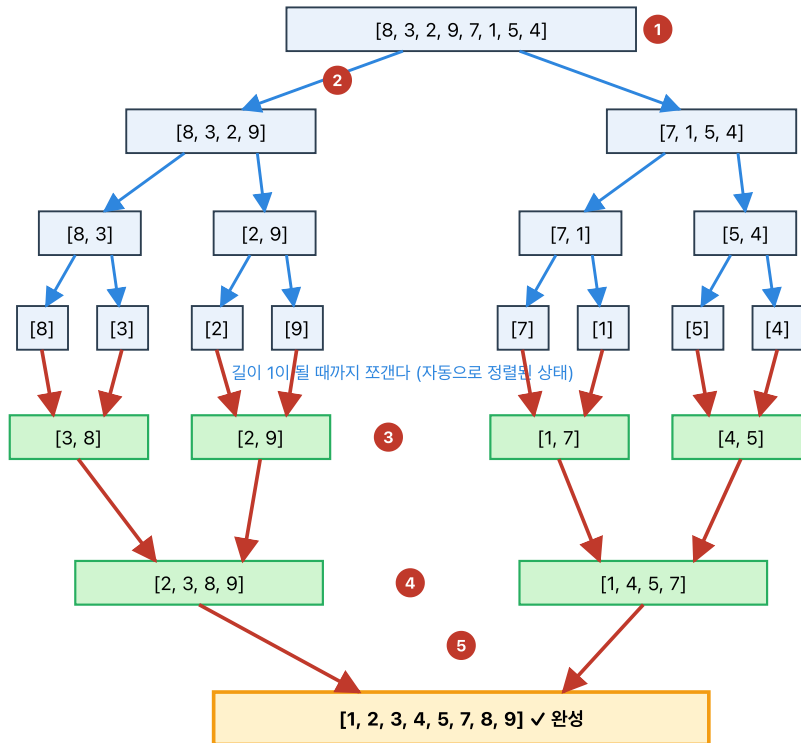
알고리즘	점화식	점근적 복잡도
이진 탐색	$T(n) = T(n/2) + O(1)$	$\Theta(\log n)$
병합 정렬	$T(n) = 2T(n/2) + O(n)$	$\Theta(n \log n)$
퀵 정렬 (평균)	$T(n) = 2T(n/2) + O(n)$	$\Theta(n \log n)$
퀵 정렬 (최악)	$T(n) = T(n-1) + O(n)$	$\Theta(n^2)$
스트라센 행렬곱	$T(n) = 7T(n/2) + O(n^2)$	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$
빠른 거듭제곱	$T(n) = T(n/2) + O(1)$	$\Theta(\log n)$

5-2. 병합 정렬

병합 정렬: Divide(분할) → Conquer(정복) → Combine(병합)

입력 배열 [8, 3, 2, 9, 7, 1, 5, 4] — 파란 화살표=분할, 빨간 화살표=병합

① Divide
(반씩 쪼갬)



② Combine
(정렬하며 병합)

병합(merge) 연산: 두 정렬된 배열 → 한 개의 정렬된 배열

- 두 배열의 맨 앞 원소를 비교 → 작은 것을 결과에 넣고 그 쪽 포인터를 한 칸 전진
- 한쪽이 끝나면 남은 쪽을 그대로 이어붙임. 한 번 병합에 $O(n)$, 레벨 수 = $\log_2 n$.

$T(n) = 2T(n/2) + n \Rightarrow \Theta(n \log n)$ (Master Theorem Case 2). 추가 메모리 $O(n)$ 필요.

반으로 쪼개 정렬한 뒤 두 정렬 리스트를 병합

- 점화식: $T(n) = 2T(n/2) + O(n) \rightarrow \Theta(n \log n)$
- 장점: 입력 상태 무관, 항상 $\Theta(n \log n)$, 안정 정렬
- 단점: 추가 배열 $O(n)$

merge_sort 의사코드 (PDF Ch5)

```
merge_sort(A[0..n-1]):
    if n > 1:
        B ← A[0..[n/2]-1] 복사
        C ← A[[n/2]..n-1] 복사
        merge_sort(B); merge_sort(C)
        merge(B, C, A)

merge(B[0..p-1], C[0..q-1], A[0..p+q-1]):
    i ← 0; j ← 0; k ← 0
    while i < p and j < q:
        if B[i] ≤ C[j]: A[k] ← B[i]; i ← i+1
        else:
            A[k] ← C[j]; j ← j+1
        k ← k+1
    남은 원소 A에 복사
```

병합 비용 $f(n) = n - 1$ 비교 (최악) → 총 비교 $\Theta(n \log n)$

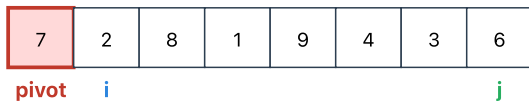
5-3. 퀵 정렬

퀵 정렬 파티션: 피벗을 기준으로 양쪽으로 분할

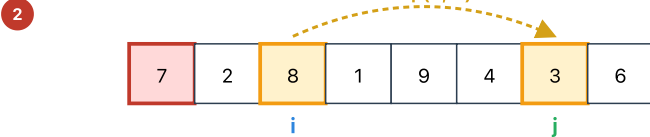
배열 [7, 2, 8, 1, 9, 4, 3, 6], 피벗 = 첫 원소 7 | i=왼쪽 포인터, j=오른쪽 포인터

초기: 피벗 7 선택

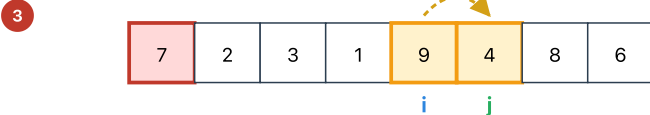
- 1 i는 왼→오로 "7보다 큰 수" 찾고, j는 오→왼로 "7보다 작은 수" 찾음



i가 전진: 8에서 멈춤 (>7). j가 후진: 3에서 멈춤 (<7) → 교환



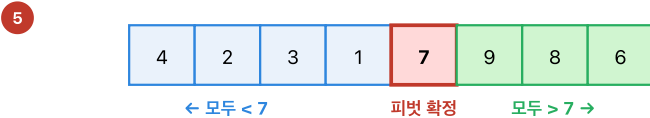
i, j 계속 이동 → i는 9(>7), j는 4(<7)에서 멈춤 → 교환



i, j 교차 (j가 i보다 왼쪽) → 피벗과 j위치를 교환 → 피벗 확정



피벗(7) ↔ A[j]=4 교환 → 피벗 7은 제자리 확정 (인덱스 4)



왼쪽, 오른쪽을 같은 방식으로 재귀 호출

복잡도 및 병합정렬과의 차이

• 평균: 최선 $O(n \log n)$ — 반씩 잘 갈릴 때 | 최악 $O(n^2)$ — 피벗이 한쪽으로만 몰릴 때 (이미 정렬된 배열 + 첫 원소 피벗)

병합정렬은 "합칠 때" 일함 / 퀵정렬은 "쪼갤 때" 이미 정렬 끝. 제자리 정렬 (추가 메모리 $O(\log n)$ 스택만)

한 번의 분할(partition) 과정 — 피벗 기준으로 작은 건 왼쪽, 큰 건 오른쪽

퀵 정렬 전체 과정 — 입력 [5,3,8,4,9,1,6,2,7] (PDF 5장 p13)

피벗은 각 부분배열의 첫 원소. 피벗 확정 후 좌/우에 같은 방식 재귀 호출 → 부분배열 크기 1 또는 0 이면 종료

1 초기 상태 — 피벗 5 선택 (첫 원소)

Level 0 호출: quicksort([5,3,8,4,9,1,6,2,7])



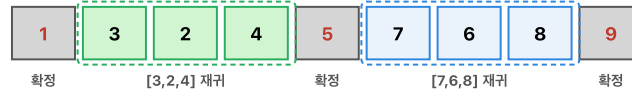
2 Level 0 파티션 완료 — 피벗 5 제자리 확정 (인덱스 4)

좌측 [1,3,2,4] : 5보다 작은 값 · 우측 [9,6,8,7] : 5보다 큰 값



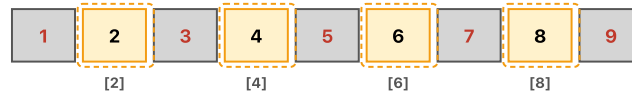
3 Level 1 파티션 완료 — 피벗 1, 9 확정

quicksort([1,3,2,4]): 피벗 1 → 왼쪽 공집합, 오른쪽 [3,2,4] / quicksort([9,6,8,7]): 피벗 9 → 왼쪽 [7,6,8], 오른쪽 공집합



4 Level 2 파티션 완료 — 피벗 3, 7 확정

quicksort([3,2,4]): 피벗 3 → [2] | 3 | [4] / quicksort([7,6,8]): 피벗 7 → [6] | 7 | [8]



5 Level 3 — 크기 1 부분배열은 이미 정렬됨 (재귀 종료)

[2], [4], [6], [8] 각각 원소 한 개 → if (l ≥ r) return; 기저 사례로 즉시 반환

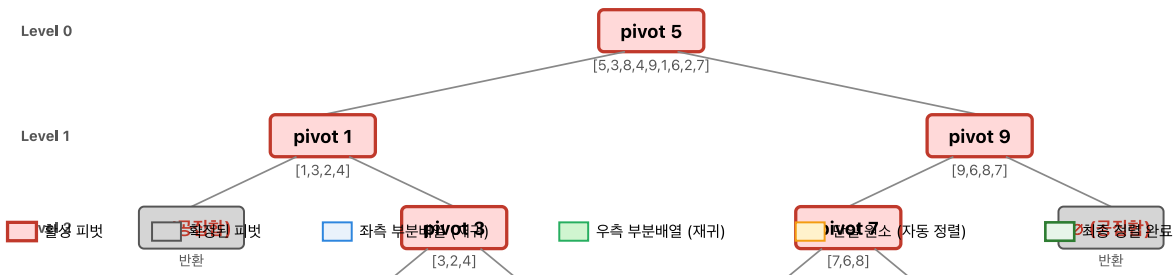


6 정렬 완료 — 모든 재귀 호출 반환

합치는 단계가 없다(in-place). 피벗 확정 = 정렬 완료



재귀 호출 트리 — 피벗으로 노드 표시, 자식은 좌/우 부분배열



전체 재귀 과정 — PDF 5장 p13 예시 [5,3,8,4,9,1,6,2,7] . 각 부분배열의 첫 원소를 피벗으로 잡아 재귀 호출하며, 피벗이 한번 확정되면 그 위치는 더 이상 이동하지 않음

상황	복잡도
평균	$O(n \log n)$
최악	$O(n^2)$ — 이미 정렬된 입력에서 항상 끝값을 피벗으로 잡는 등 쓸릴 때

회피 기법: 랜덤 피벗, 중앙값-3(median-of-three)

Hoare 분할(partition) 의사코드 (PDF Ch5)

```
partition(A[l..r]):
    pivot ← A[l]
    i ← l; j ← r+1
    repeat
        repeat i←i+1 until A[i] ≥ pivot
        repeat j←j-1 until A[j] < pivot
        swap A[i] and A[j]
    until i ≥ j
    swap A[l] and A[j]
```

```

repeat j ← j - 1 until A[j] ≤ pivot
  swap(A[i], A[j])
until i ≥ j
swap(A[i], A[j])      # 마지막 교환 되돌리기
swap(A[l], A[j])      # 피벗을 제자리에
return j

```

피벗=기준값 — 필기 강조

🔗 퀵정렬 점화식 정밀 (PDF Ch5)

- 최선/평균: 피벗이 중앙 근처 $\rightarrow T(n) = 2T(n/2) + n \Rightarrow \Theta(n \log n)$
- 최악: 이미 정렬된 입력, 피벗이 항상 끝값 $\rightarrow$ 한쪽 0개, 다른 쪽 $n - 1$ 개

$$T_{\text{worst}}(n) = T(n-1) + T(0) + (n-1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = \Theta(n^2)$$

5-4. 이진트리 순회

순회	순서	첫 원소 의미
전위 Pre-order	V L R	전체 트리의 루트
중위 In-order	L V R	—
후위 Post-order	L R V	마지막이 루트

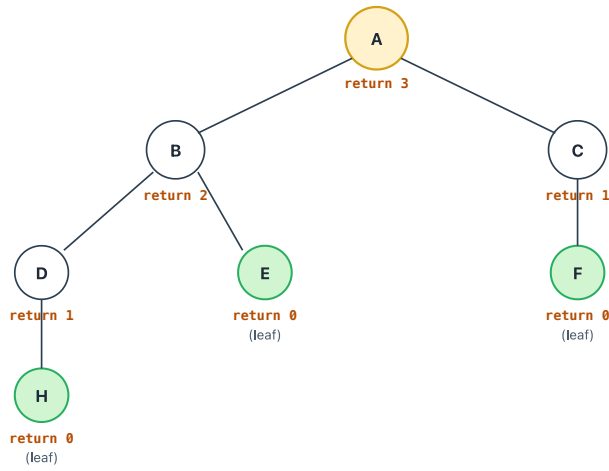
🔄 트리 복원

전위+중위 또는 중위+후위 $\rightarrow$ 복원 가능
 전위+후위만으로는 일반적으로 불가능

☞ 트리 높이의 재귀 계산 — 분할정복 전형

$\text{height}(v) = 1 + \max(\text{height}(v.\text{left}), \text{height}(v.\text{right}))$, 빈 트리는 -1
 각 노드를 정확히 1회씩 방문 $\rightarrow \Theta(n)$

이진트리 높이 = $1 + \max(\text{left}, \text{right})$ — 분할정복 재귀



노드별 계산 (후위 순회)

⊙ H (leaf)
 $\text{height}(H) = 0 \leftarrow \text{기저}$

⊙ D
 $\text{height}(D) = 1 + \max(0, -\infty) = 1 + 0 = 1$

⊙ E, F (leaves)
 $\text{height}(E) = 0, \text{height}(F) = 0$

⊙ B
 $\text{height}(B) = 1 + \max(1, 0) = 2$

⊙ C
 $\text{height}(C) = 1 + \max(0, -\infty) = 1$

⊙ A (root)
 $\text{height}(A) = 1 + \max(2, 1) = 3$

※ 빈 자식의 높이를 -1로 정의하면
 $\text{height}(\text{leaf}) = 1 + \max(-1, -1) = 0$ 로 통일

의사코드 & 점화식

```

height(v):
if v == null: return -1
return 1 + max(height(v.left), height(v.right))

```

점화식: $T(n) = T(k) + T(n-1-k) + O(1) \Rightarrow T(n) = \Theta(n)$ (전체 노드 1회씩)

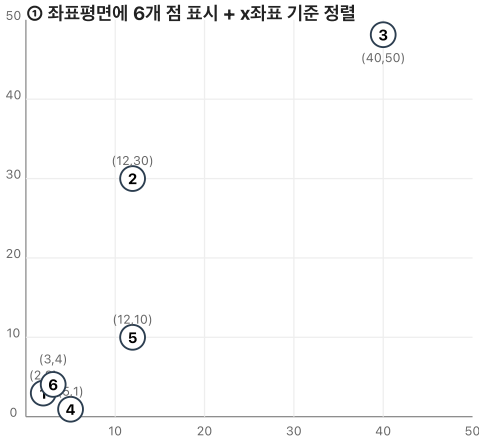
5-5. 최근접 쌍

방법	복잡도
완전탐색 (3장)	$O(n^2)$
분할정복 (5장)	$O(n \log n)$

핵심 아이디어: 좌/우에서 각각 최근접 쌍 → 경계 주변의 띠(strip)만 추가 검사 — 띠 너비 = 지금까지의 최소 거리 d

최근접 쌍 — 분할정복 예시 (PDF 5장 p27)

입력: $P = \{(2,3), (12,30), (40,50), (5,1), (12,10), (3,4)\}$ — 모든 쌍 중 가장 가까운 두 점 찾기



② x좌표 정렬 + 중앙 분할

정렬 후 앞 절반 P_L , 뒷 절반 P_R 로 분할 $\min x \approx 8.5$

P1 (2,3)	P6 (3,4)	P4 (5,1)	P2 (12,30)	P5 (12,10)	P3 (40,50)
P_L (왼쪽 3개)			P_R (오른쪽 3개)		

③-좌 Conquer ($n \leq 3$ 완전탐색)

$d(P_1, P_6) = \sqrt{((3-2)^2 + (4-3)^2)} = \sqrt{1+1} = \sqrt{2} \approx 1.414$
 $d(P_1, P_4) = \sqrt{(9+4)} = \sqrt{13} \approx 3.606$
 $d(P_6, P_4) = \sqrt{(4+9)} = \sqrt{13} \approx 3.606$
 $\rightarrow d_L = \sqrt{2}$ (쌍: P1 ↔ P6)

③-우 Conquer ($n \leq 3$ 완전탐색)

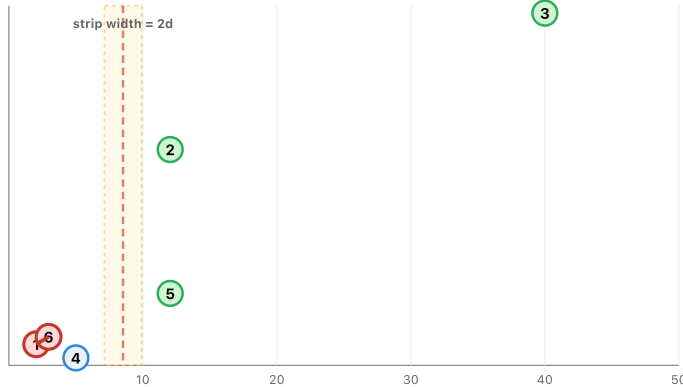
$d(P_2, P_5) = \sqrt{(0+400)} = 20$
 $d(P_2, P_3) = \sqrt{(784+400)} \approx 34.41$
 $d(P_5, P_3) = \sqrt{(784+1600)} \approx 48.83$
 $\rightarrow d_R = 20$ (쌍: P2 ↔ P5)

$d = \min(d_L, d_R) = \min(\sqrt{2}, 20) = \sqrt{2}$

(측 정렬 순서: P1(2,3) → P6(3,4) → P4(5,1) → P2(12,30) → P5(12,10) → P3(40,50))

④ Combine — 경계 주변 "띠(strip)" 내부만 추가 검사

기준선 $x=8.5$ 앞쪽에서 $d=\sqrt{2} \approx 1.414$ 만큼만 후보 존재 가능 → 범위: $x \in [7.09, 9.91]$



띠 안에 어떤 점도 없음 → 경계 교차 쌍 검사 불필요 → 최종 답 = $d_L = \sqrt{2}$

⑤ 최종 결과

최근접 쌍 = P1(2,3) ↔ P6(3,4)

최소 거리 = $\sqrt{2} \approx 1.4142$

분할정복 3단계 요약

Divide: x좌표 중앙값 기준으로 P_L (3개) / P_R (3개) 분할

Conquer: 각 부분 재귀 ($n \leq 3 \rightarrow$ 완전탐색)
 $d_L = \sqrt{2}, d_R = 20$

Combine: 띠 내부만 추가 검사
이 예시는 띠가 비어있음 → $d = \min(d_L, d_R) = \sqrt{2}$

핵심 Lemma — 왜 띠만 한 점은 "최대 7개"와만 비교하면 되는가?

- 좌/우 각 그룹 내부의 모든 쌍 거리는 이미 $\geq d$ 임이 보장됨 → 한 번 d 의 격자 박스 하나에 한 점만 들어갈 수 있음
- 띠는 너비 $2d$. 한 점 기준 "아래쪽 y차이 $\leq d$ " 영역은 $2d \times d$ 직사각형 → 여기 들어갈 수 있는 점은 기하학적으로 최대 7개
- 따라서 띠 내 각 점에서 교차 상수(7)회만 비교 → Combine 단계 $O(n)$ → 전체 $T(n) = 2T(n/2) + O(n) = O(n \log n)$

PDF 5장 p27 예시 — 6개 점 직접 풀이

입력: $P = \{P_1(2,3), P_2(12,30), P_3(40,50), P_4(5,1), P_5(12,10), P_6(3,4)\}$

① Divide — x좌표 정렬 후 중앙 분할

- 정렬 순서: $P_1(2,3) \rightarrow P_6(3,4) \rightarrow P_4(5,1) \parallel P_2(12,30) \rightarrow P_5(12,10) \rightarrow P_3(40,50)$
- $P_L = \{P_1, P_6, P_4\}, P_R = \{P_2, P_5, P_3\}$, 경계 $x \approx 8.5$

② Conquer — 각 부분 완전탐색 ($n \leq 3$ 이므로 기저 사례)

- 왼쪽: $d(P_1, P_6) = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414 \leftarrow$ 최소
- 오른쪽: $d(P_2, P_5) = \sqrt{0 + 400} = 20 \leftarrow$ 최소
- $d = \min(d_L, d_R) = \min(\sqrt{2}, 20) = \sqrt{2}$

③ Combine — 띠(strip) 범위: $x \in [8.5 - \sqrt{2}, 8.5 + \sqrt{2}] \approx [7.09, 9.91]$

- 이 범위 안에 들어오는 점이 없음 → 경계 교차 쌍 검사 불필요

최종 답: $P_1(2,3) \leftrightarrow P_6(3,4)$, 거리 $\sqrt{2} \approx 1.4142$

🔗 최근접 쌍 복잡도 유도 (PDF Ch5)

- 점화식: $T(n) = 2T(n/2) + f(n)$

- 합치기: y좌표 정렬이 매번 필요하면 $f(n) = n \log n \rightarrow T(n) = \Theta(n(\log n)^2)$
- 사전 y정렬을 한 번만 해두고 재사용 $\rightarrow f(n) = n \rightarrow \Theta(n \log n)$
- 띠 영역 내 한 점이 비교하는 대상은 최대 7개 점($\leq 2d \times d$ 직사각형 내 기하학적 상한)

5-6. 스트라센 행렬 곱셈

방법	곱셈 횟수	복잡도
표준	8회	$O(n^3)$
스트라센	7회	$O(n^{\log_2 7}) \approx O(n^{2.807})$

$$T(n) = 7T(n/2) + O(n^2)$$

스트라센 행렬 곱셈 — 2x2 예시 (곱셈 8회 → 7회)

$A \times B = C$ 를 두 가지 방법으로 계산하여 곱셈 연산 횟수를 직접 비교

① 입력 행렬

A		B		C = A × B	
1 _a	2 _b	5 _e	6 _f	19	22
3 _c	4 _d	7 _g	8 _h	43	50

② 표준 방식 — 곱셈 8회 + 덧셈 4회

공식: $C_{ij} = A_{i1} \cdot B_{1j} + A_{i2} \cdot B_{2j}$

$$C_{11} = a \cdot e + b \cdot g = 1 \cdot 5 + 2 \cdot 7 = 5 + 14 = 19 \quad (\text{곱셈 2회})$$

$$C_{12} = a \cdot f + b \cdot h = 1 \cdot 6 + 2 \cdot 8 = 6 + 16 = 22 \quad (\text{곱셈 2회})$$

$$C_{21} = c \cdot e + d \cdot g = 3 \cdot 5 + 4 \cdot 7 = 15 + 28 = 43 \quad (\text{곱셈 2회})$$

$$C_{22} = c \cdot f + d \cdot h = 3 \cdot 6 + 4 \cdot 8 = 18 + 32 = 50 \quad (\text{곱셈 2회})$$

총 곱셈 연산: $2+2+2+2 = 8$ 회

블록 크기 n이면 $T(n)=8 \cdot T(n/2)+O(n^2) \rightarrow O(n^3)$

③ 기억 요령 — M₁~M₇ 외우는 대칭 규칙

- M₁: 대각합 × 대각합 — (a+d)(e+h) (양쪽 모두 "합")
- M₂: 아래행합 × 좌상 — (c+d)·e
- M₃: 좌상 × 우열차 — a(f-h) — 우상-우하
- M₄: 우하 × 좌열차 — d(g-e) — 좌하-좌상
- M₅: 위행합 × 우하 — (a+b)·h
- M₆: 아래-위대각 × 위행합 — (c-a)(e+f)
- M₇: 위-아래대각 × 아래행합 — (b-d)(g+h)

③ 스트라센 방식 — 곱셈 7회 + 덧셈/뺄셈 18회

덧·뺄셈은 늘지만, 곱셈이 8→7로 줄어드는 것이 핵심 (큰 n에서 압도적 이득)

$$M_1 = (a+d)(e+h) = (1+4)(5+8) = 5 \cdot 13 = 65$$

$$M_2 = (c+d) \cdot e = (3+4) \cdot 5 = 7 \cdot 5 = 35$$

$$M_3 = a \cdot (f-h) = 1 \cdot (6-8) = 1 \cdot (-2) = -2$$

$$M_4 = d \cdot (g-e) = 4 \cdot (7-5) = 4 \cdot 2 = 8$$

$$M_5 = (a+b) \cdot h = (1+2) \cdot 8 = 3 \cdot 8 = 24$$

$$M_6 = (c-a)(e+f) = (3-1)(5+6) = 2 \cdot 11 = 22$$

$$M_7 = (b-d)(g+h) = (2-4)(7+8) = (-2) \cdot 15 = -30$$

결과 블록 조합 (덧셈/뺄셈만, 곱셈 없음)

$$C_{11} = M_1 + M_4 - M_5 + M_7 = 65 + 8 - 24 + (-30) = 19 \quad \checkmark$$

$$C_{12} = M_3 + M_5 = (-2) + 24 = 22 \quad \checkmark$$

$$C_{21} = M_2 + M_4 = 35 + 8 = 43 \quad \checkmark$$

$$C_{22} = M_1 + M_3 - M_2 + M_6 = 65 + (-2) - 35 + 22 = 50 \quad \checkmark$$

총 곱셈 연산: 7회 (M₁~M₇ 각 1회)

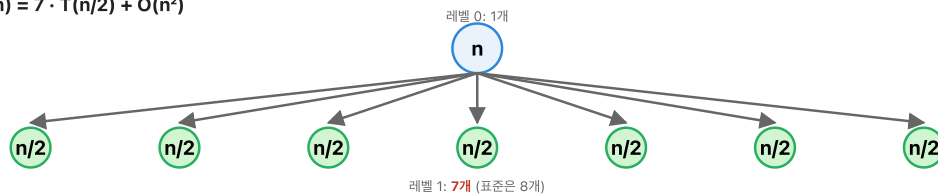
$$T(n) = 7 \cdot T(n/2) + O(n^2)$$

$$\rightarrow O(n^{\log_2 7}) \approx O(n^{2.807})$$

③ 곱셈 8회 vs 7회 — 큰 n에서 얼마나 차이 나는가?

- n=2일 때: 표준 8회 vs 스트라센 7회 — 차이는 거의 없음 (덧셈 오버헤드 때문에 오히려 느릴 수도)
- 재귀: 각 레벨에서 8→7로 줄면 n=1024 기준 표준 $\approx 10^9$, 스트라센 $\approx 7^{10} \approx 2.8 \times 10^8$ — 약 3배 이상 빠름
- 지수: $\log_2 8 = 3$ vs $\log_2 7 \approx 2.807$ — 겨우 0.19 차이지만 n이 클수록 차이가 지수적으로 벌어짐
- 핵심 교훈: "곱셈 1회 줄이기"는 단순히 1/8 절약이 아니다. 재귀 트리 전체에 걸쳐 복잡도 지수 자체를 낮추는 효과.

③ 재귀 트리 — $T(n) = 7 \cdot T(n/2) + O(n^2)$



레벨 1: 7개 (표준은 8개)

$$\text{레벨 } k: 7^k \text{ 개} \rightarrow \text{전체 호출 수} = 1 + 7 + 49 + \dots + 7^{\log_2 n} = O(7^{\log_2 n}) = O(n^{\log_2 7}) \approx O(n^{2.807})$$

$$\text{숫자 예시} \rightarrow A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}, C = AB = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

표준 방식 (곱셈 8회)

- $C_{11} = 1 \cdot 5 + 2 \cdot 7 = 19$, $C_{12} = 1 \cdot 6 + 2 \cdot 8 = 22$
- $C_{21} = 3 \cdot 5 + 4 \cdot 7 = 43$, $C_{22} = 3 \cdot 6 + 4 \cdot 8 = 50$

스트라센 $M_1 \sim M_7$ (곱셈 7회)

M_i	식	계산	값
M_1	$(a + d)(e + h)$	$(1 + 4)(5 + 8) = 5 \cdot 13$	65
M_2	$(c + d)e$	$(3 + 4) \cdot 5 = 7 \cdot 5$	35
M_3	$a(f - h)$	$1 \cdot (6 - 8) = 1 \cdot (-2)$	-2
M_4	$d(g - e)$	$4 \cdot (7 - 5) = 4 \cdot 2$	8
M_5	$(a + b)h$	$(1 + 2) \cdot 8 = 3 \cdot 8$	24
M_6	$(c - a)(e + f)$	$(3 - 1)(5 + 6) = 2 \cdot 11$	22
M_7	$(b - d)(g + h)$	$(2 - 4)(7 + 8) = (-2) \cdot 15$	-30

결과 블록 재조합 (곱셈 0회, 덧셈·뺄셈만)

- $C_{11} = M_1 + M_4 - M_5 + M_7 = 65 + 8 - 24 - 30 = \mathbf{19} \checkmark$
- $C_{12} = M_3 + M_5 = -2 + 24 = \mathbf{22} \checkmark$
- $C_{21} = M_2 + M_4 = 35 + 8 = \mathbf{43} \checkmark$
- $C_{22} = M_1 + M_3 - M_2 + M_6 = 65 - 2 - 35 + 22 = \mathbf{50} \checkmark$

 스트라센 **M1~M7** 공식 정리 — 2×2 블록 $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, $B = \begin{pmatrix} e & f \\ g & h \end{pmatrix}$

$$M_1 = (a + d)(e + h)$$

$$M_2 = (c + d)e$$

$$M_3 = a(f - h)$$

$$M_4 = d(g - e)$$

$$M_5 = (a + b)h$$

$$M_6 = (c - a)(e + f)$$

$$M_7 = (b - d)(g + h)$$

결과 블록:

$$C = \begin{pmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{pmatrix}$$

- 표준: 블록 곱셈 8회 + 덧셈 4회
- 스트라센: **곱셈 7회** + 덧셈/뺄셈 18회 (덧셈 늘고 곱셈 줄어듦)
- 점화식: $T(n) = 7T(n/2) + O(n^2) \rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$ (결과만 암기)

 왜 "곱셈 1회 줄이기"가 중요한가?

큰 n 일수록 재귀 트리 전체에 걸쳐 복잡도 지수 자체가 낮아짐.

- $n = 1024$ 기준: 표준 $\approx 10^9$ 회 vs 스트라센 $\approx 2.8 \times 10^8$ 회 (약 3배 이상 빠름)
- 지수 차이 $\log_2 8 - \log_2 7 \approx 0.193$ 은 작아 보이지만, $n^{0.193}$ 은 n 이 클수록 빠르게 커짐

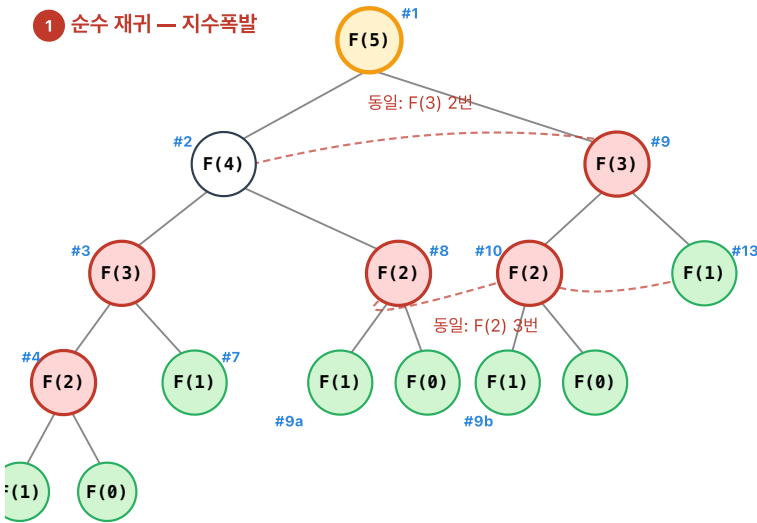
 경고 — 대규모 실수 행렬에서 수치 오차 누적 위험

5-7. 피보나치 재귀의 함정

피보나치 재귀 호출 트리: F(5) — 중복 부분문제와 메모이제이션

규칙: $F(n) = F(n-1) + F(n-2)$, $F(0)=0$, $F(1)=1$. 파란 숫자는 호출 순서(DFS).

1 순수 재귀 — 지수폭발



2 메모이제이션 — 한 번 계산, 재사용

i	0	1	2	3	4	5
memo	0	1	1	2	3	5

- $F(0)$, $F(1)$: 베이스 → 즉시 반환
- $F(2) = F(1) + F(0) = 1$ (한 번만 계산, 캐시)
- $F(3)$ 두번째 호출부터는 **O(1)** 테이블 조회
- 총 호출: $n+1$ 번 → 시간 $O(n)$, 공간 $O(n)$

상향식 DP 의사코드

```
memo[0]=0; memo[1]=1;
for i = 2 to n:
    memo[i] = memo[i-1] + memo[i-2]
return memo[n]
```

호출 횟수 정밀 분석 — F(5)

- 순수 재귀: $F(5)=1$, $F(4)=1$, $F(3)=2$, $F(2)=3$, $F(1)=5$, $F(0)=3$ → 총 15 호출
- 메모이제이션: $F(0) \sim F(5)$ 각 1회씩 → 총 6 호출 (테이블 조회는 비재귀)
- 일반화: $T_{naive}(n) \approx F(n+1) \approx \phi^n / \sqrt{5}$ (지수), $T_{dp}(n) = O(n)$ (선형)

성장을 체감 — n이 커지면?

n	순수 재귀 호출수	메모 호출수	격차	현실 시간(1ns/호출 가정)
10	177	11	16배	$< 1\mu s$ vs $< 1\mu s$
30	2,692,537	31	약 87,000배	2.7ms vs 31ns
50	$\sim 4 \times 10^{10}$	51	약 8억배	$\sim 40초$ vs $51ns$
100	$\sim 3.5 \times 10^{20}$	101	사실상 ∞	$\sim 11,000년$ vs $101ns$

$F(n) = F(n-1) + F(n-2)$ 를 그대로 재귀 호출 → 같은 부분 문제가 계속 반복
→ $O(\varphi^n)$ (지수)

교훈

부분 문제가 서로 겹치면 분할정복은 비효율
→ **7장 DP(memoization)**로 해결

피보나치 4가지 알고리즘 비교 (PDF Ch5-Ch6 종합)

방식	시간	공간	특징
분할정복 재귀	$O(\varphi^n) \approx O(2^n)$	$O(n)$ (스택)	중복 계산 폭발
반복 (bottom-up)	$O(n)$	$O(1)$	2개 변수만 유지
행렬 거듭제곱	$O(\log n)$	$O(1)$	$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$
테이블(메모이제이션)	$O(n)$	$O(n)$	공간으로 시간 벌기 (Ch6)

점화식 $T(n) = T(n-1) + T(n-2) + 1 \rightarrow$ 해 $O(\varphi^n)$, $\varphi = (1 + \sqrt{5})/2 \approx 1.618$

6. 공간으로 시간 벌기 Space-Time Tradeoff (Ch6)

6장의 본질 — "메모리를 더 써서 시간을 산다"

미리 계산한 결과를 테이블에 저장 → 다음 번부터는 계산 없이 꺼내 쓴다

- 비유: 족보·기출 풀이집 — 시험장에서 매번 처음부터 풀지 않고 외운 답을 바로 씀
- 일상: 전화번호부·주소록 — 이름 → 번호 탐색을 $O(1)$ 에 가깝게
- 핵심 장치 4종: ① 카운트 배열, ② 해시 테이블, ③ 시프트 테이블(Horspool), ④ 전처리 인덱스

6-1. 카운팅 정렬

계수 정렬: 비교 없이 "몇 번 나왔는지"만 세면 $O(n+k)$

입력 [3,1,4,1,5,9,2,6,5,3,5] — 값 범위 0~9, $n=11$, $k=10$

- 1 입력 배열 A — 그대로 한 번 훑는다

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

"몇 번?" 카운트

- 2 Count 배열 — 각 값의 출현 횟수

$v=0$	$v=1$	$v=2$	$v=3$	$v=4$	$v=5$	$v=6$	$v=7$	$v=8$	$v=9$
0	2	1	2	1	3	1	0	0	1

5가 3번 등장 ✓

"이하 몇 개?" 누적

- 3 누적합 배열 — $C[v]$ 는 "값 v 이하"의 개수 (= v 의 끝자리 인덱스+1)

0	2	3	5	6	9	10	10	10	11
---	---	---	---	---	---	----	----	----	----

$C[5]=9 \rightarrow$ "5 이하 값이 9개" $\rightarrow$ 마지막 5는 output[8] 위치에 배치

A를 뒤 $\rightarrow$ 앞 스캔하며 배치 (stable)

- 4 출력 배열 — 누적합을 인덱스로 사용

1	1	2	3	3	4	5	5	5	6	9
0	1	2	3	4	5	6	7	8	9	10

정렬 완료 ✓

복잡도 $O(n+k)$, 비교-없는 정렬 $\Rightarrow \Omega(n \log n)$ 하한을 벗어남

조건: 값이 작은 정수. $k \gg n$ 이면 비효율. 안정(stable) — 누적합으로 뒤부터 배치하면 원순서 보존

값별 개수를 세어 누적합으로 자리를 정함

비유: 투표 집계 — 후보(값)별 표를 세고, 누적으로 "몇 등까지 몇 명" 계산

단계:

1. Count: 값별 빈도
2. 누적합: "값 v 이하의 개수"
3. 출력: 원본을 뒤에서 앞으로 배치 (안정성 확보)

복잡도: $O(n+k)$ — k 가 n 보다 너무 크면 오히려 비효율

counting_sort 의사코드 (PDF Ch6)

```
counting_sort(A[0..n-1], k):      # 값 범위 0..k
    for i ← 0 to k: count[i] ← 0
    for j ← 0 to n-1: count[A[j]] ← count[A[j]] + 1
    for i ← 1 to k: count[i] ← count[i] + count[i-1]    # 누적합
    for j ← n-1 downto 0:                                # 뒤에서 앞으로 (안정)
        B[count[A[j]]-1] ← A[j]
        count[A[j]] ← count[A[j]] - 1
    return B
```

- 누적합의 의미: $\text{count}[i]$ = "값이 i 이하인 원소의 개수" = 값 i 의 출력 배열에서 끝 위치+1
- 뒤에서 앞으로 배치해야 같은 값이 입력 순서를 유지 → **안정 정렬**

6-2. 기수 정렬

Ch.6 예제 — 기수정렬(Radix Sort / LSD) : [28, 93, 39, 81, 62, 72, 47, 15]

자리수별로 안정 정렬 반복. LSD = 낮은 자리부터 → 높은 자리까지. 총 d 번 pass (d = 최대 자리수)

1 원본 배열 (2자리 수이므로 $d = 2$ — 2회 pass 필요)

28	93	39	81	62	72	47	15
----	----	----	----	----	----	----	----

청=10자리 빨=1자리

2 Pass 1 — 1의 자리 기준 버킷 분배 & 수집

버킷	0	1 → 81	2 → 62, 72	3 → 93	4	5 → 15	6	7 → 47	8 → 28	9 → 39
----	---	--------	------------	--------	---	--------	---	--------	--------	--------

수집 (버킷 0→9 순서, FIFO 유지 — 안정성 보장):

81	62	72	93	15	47	28	39
----	----	----	----	----	----	----	----

1의 자리 기준 정렬됨

3 Pass 2 — 10의 자리 기준 버킷 분배 & 수집

버킷	0	1 → 15	2 → 28	3 → 39	4 → 47	5	6 → 62	7 → 72	8 → 81	9 → 93
----	---	--------	--------	--------	--------	---	--------	--------	--------	--------

핵심: 같은 버킷 내에서는 입력 순서 유지 — Pass 1에서 정리한 1의 자리 순서가 보존됨

15	28	39	47	62	72	81	93
----	----	----	----	----	----	----	----

완전 정렬 완료 ✓

4 안정 정렬이 필수인 이유 — 62와 72 비교

Pass 1 후: ..., 62, 72, ... (둘 다 1자리 2, 입력 순서대로)

Pass 2 버킷 6에는 62만, 버킷 7에는 72만 → $6 < 7$ 이므로 자연스럽게 62가 먼저

만약 같은 10자리(예: 62와 63 모두 6)라면:

Pass 1에서 확립된 "1자리가 작은 것이 앞" 순서가 유지되어야 함. 불안정 정렬 쓰면 붕괴 → 오답!

5 복잡도 & 제약

$$T(n) = O(d \cdot (n + k))$$

• d = 자리수 (여기선 2) • k = 기수 (여기선 10) • n = 원소 개수 (여기선 8)

• d, k 가 상수면 → 선형 시간 $O(n)$. 비교 기반 $O(n \log n)$ 하한을 우회!

단점: 메모리 $O(n+k)$, 키가 정수/고정폭 문자열일 때만 적용 가능

낮은 자릿수부터 안정 정렬을 반복

$$O(d(n + r))$$

🔗 안정성 필수

각 단계가 안정 정렬이어야 한다 — 안정성이 깨지면 낮은 자릿수의 순서가 손상

→ 보조 루틴으로 카운팅 정렬이 자주 쓰임

기수정렬 단계별 예제 (PDF Ch6) — 입력 28, 93, 39, 81, 63, 72, 38, 26, 16, 18, 62

1단계: 일의 자리로 10개 버킷 배분

1: 81, 2: 72, 62, 3: 93, 63, 6: 26, 16, 8: 28, 38, 18, 9: 39

모으기 → 81, 72, 62, 93, 63, 26, 16, 28, 38, 18, 39

2단계: 십의 자리로 재배분 (위 결과 기준)

1: 16, 18, 2: 26, 28, 3: 38, 39, 6: 62, 63, 7: 72, 8: 81, 9: 93

모으기 → 16, 18, 26, 28, 38, 39, 62, 63, 72, 81, 93 (정렬 완료)

• $d = 2$ 자리, $r = 10$ 버킷 → $O(d(n + r)) = O(2 \cdot 21)$

6-3. Horspool 문자열 매칭

Horspool 문자열 매칭: 패턴 "BARBER" 찾기

텍스트: JIM_SAW_ME_IN_A_BARBERSHOP | 핵심: 오른쪽 끝부터 비교 → 불일치면 shift표로 크게 점프

- 1 shift 표 만들기 — 패턴 "BARBER" (m=6)
규칙: 마지막 문자 제외, 오른쪽에서의 거리. 기타(패턴에 없는) = m = 6

문자	B	A	R	E	기타
shift	4	2	3	1	6

B·A·R·B·E·R에서 마지막 R 제외
B는 끝에서 4칸, A는 2칸, R은 3칸, E는 1칸
B가 두 번이면 더 가까운 쪽(오른쪽)

- 2 매칭 시도 (각 시도: 오른쪽 끝 비교 → shift 결정)

시도 1: 끝자리 W vs R 불일치. W는 기타 → 6칸 이동

J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
 B A R B E R

시도 2: 끝자리 _ vs R 불일치. _는 기타 → 6칸 이동

J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
 B A R B E R

시도 3: 끝자리 A vs R 불일치. A는 shift[A]=2 → 2칸 이동

J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
 B A R B E R

시도 4: 끝 R vs R 일치! 왼쪽으로 E·B·R·A·B 모두 일치 → 발견 ✓

J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
 B A R B E R

복잡도 & 핵심 아이디어
평균 $\Theta(n)$, 최악 $\Theta(nm)$. 전처리 $\Theta(m+\sigma)$. KMP와 달리 "오른쪽 끝부터" 비교 → 불일치 문자로 큰 점프 가능

패턴을 오른쪽 끝부터 비교, 불일치 시 shift 표로 한 번에 여러 칸 점프

shift 표 만드는 규칙 (패턴의 마지막 문자 제외한 나머지):

$shift(c) = \text{패턴 오른쪽 끝에서 } c \text{가 처음 나오는 거리}$

패턴에 없는 문자 → shift = 패턴 길이 m

예: 패턴 BARBER, 텍스트 JIM_SAW_ME_IN_A_BARBERSHOP
→ shift: B=4, A=2, R=3, E=1, 기타=6

구분	복잡도
평균	$O(n)$
최악	$O(nm)$

Horspool 3대 암기

- 비교 방향: 오른쪽 → 왼쪽
- 이동량: 불일치 난 텍스트 문자(정렬된 위치의 문자) 기준
- 전처리: $\Theta(m + \Sigma)$

shift table 공식 정밀 (PDF Ch6)

패턴 $P[0..m-1]$, 마지막 문자 $P[m-1]$ 제외하고 작성:

$$t(c) = \begin{cases} m-1-j & c = P[j], j \text{는 } P[0..m-2] \text{에서 } c \text{의 가장 오른쪽 위치} \\ m & c \text{가 } P[0..m-2] \text{에 없음} \end{cases}$$

예: $P = \text{BARBER}$ ($m = 6$). 앞 5글자 BARBE 에서

- B : 인덱스 2가 오른쪽 끝 → $t(B) = 6 - 1 - 2 = 3 \dots$ 재검토

- 실제 교재 표기: "오른쪽 끝에서 세어" $\rightarrow t(B) = 3, t(A) = 4, t(R) = 2, t(B) = 3, t(E) = 1$
- 마지막 문자 R 은 표에서 제외. 표 읽는 법: **(패턴 길이-1) - (마지막 등장 인덱스)**

보너스: Boyer-Moore 원형

Ch.6 예제 — 보이어-무어(Boyer-Moore) : 두 휴리스틱 조합

오른쪽 끝부터 비교. **불일치 문자(Bad Character)**와 **좋은 서픽스(Good Suffix)** 중 더 큰 shift 선택

1 휴리스틱 ① 불일치 문자 (Bad Character) — Horspool과 동일 원리

텍스트 T = "...A..." 에서 패턴 P = "BARBER"의 마지막 R이 A와 불일치

B	A	R	B	E	R
---	---	---	---	---	---

↑ 비교 실패: R ≠ A

규칙: 불일치된 T 문자 c의 "패턴 내 가장 오른쪽 위치"까지 맞추도록 shift (없으면 패턴 길이만큼)

주의: 불일치 위치가 c의 마지막 출현보다 왼쪽이면 역방향 shift가 될 수 있음 $\rightarrow \max(1, t_1(c))$ 사용

2 휴리스틱 ② 좋은 서픽스 (Good Suffix) — 이미 일치한 꼬리를 활용

텍스트 일부 "...ER..." 가 패턴 끝 "ER"와 일치 후, 그 앞 글자에서 불일치

T:	?	X	E	R
P:	B	B	E	R

꼬리 "ER" 일치, 그 앞 불일치

규칙: 일치한 서픽스 S를 패턴의 왼쪽에서 다시 찾아 맞추도록 shift (없으면 일치하는 접두가 서픽스가 되도록)

- 패턴 "BARBER"의 서픽스 "ER"은 왼쪽에 없음 $\rightarrow$ 대체 규칙: 서픽스의 일부가 접두어와 일치하는지 확인
- 패턴 길이만큼 점프 가능 (가장 큰 이득)
- 선계산표 $t_2[k]$: "길이 k의 서픽스가 일치한 뒤 불일치 시" 이동량

3 조합 규칙 — 두 휴리스틱 중 더 큰 shift를 선택

상황	$t_1(c)$ — 불일치 문자	$t_2[k]$ — 좋은 서픽스	최종 shift
$k = 0$ (즉시)	$\max(1, t_1(c))$	(해당 없음)	$\max(1, t_1(c))$
$k \geq 1$ (꼬리 일치)	$\max(1, t_1(c) - k)$	$t_2[k]$	$\max\{\max(1, t_1(c) - k), t_2[k]\}$

4 복잡도와 비교

Brute Force: 최악 $\theta(nm)$, 평균 $O(n)$ — 매번 1칸 shift

Horspool: 최악 $\theta(nm)$, 평균 $\theta(n)$ — 불일치 문자만 사용, 단순

Boyer-Moore: 최악 $\theta(n)$, 평균 $\theta(n/m)$ — 두 휴리스틱 결합, 큰 패턴에 유리

핵심 통찰: 패턴이 길수록 ($m \uparrow$) 한 번 shift로 건너뛰는 문자가 많아짐 $\rightarrow$ 텍스트의 일부만 검사

실용: 긴 패턴 ($m \geq 10$) & 큰 알파벳(영문 텍스트)에서 최고 성능 — **grep**, 에디터 찾기 기본 알고리즘

Horspool의 상위 알고리즘. 두 개의 시프트 표를 병행 사용:

- 나쁜 문자(Bad Character) 규칙: Horspool과 동일한 기준의 점프표
- 좋은 접미부(Good Suffix) 규칙: 이미 일치한 접미부 정보를 활용해 더 크게 점프

두 규칙 중 더 큰 점프를 선택 $\rightarrow$ 평균 성능이 Horspool보다 우수. 단 구현·전처리 비용↑.

🔗 Boyer-Moore 두 휴리스틱 결합 공식 (PDF Ch6)

불일치 시 이동량:

$$d = \max(d_1, d_2)$$

- d_1 = **bad symbol shift** (Horspool과 유사하나 이미 일치한 접미부 길이 k 를 반영): $d_1 = \max(t_1(c) - k, 1)$
- d_2 = **good suffix shift**: 일치한 접미부 $P[i + 1..m - 1]$ 이 패턴의 다른 위치에서 재등장하는 거리 (없으면 m)
- 완전일치($k = m - 1$) 시에는 good suffix만 사용
- 최악 $O(nm)$, 평균 $O(n/m)$ (패턴이 길수록 빠름) $\leftarrow$ Horspool보다 우수

6-4. 해싱

"키에 대해 주소를 계산해 바로 접근" $\rightarrow$ 이상적 $O(1)$

🔗 한 문단으로 해싱 — "사물함 번호표"

- 이름을 공식에 넣으면 사물함 번호가 바로 튀어나온다 → 굳이 찾아다닐 필요 없음
- 적재율 $\alpha = n/m = \$ (넣은 물건 수) \div (사물함 칸 수) — "얼마나 찼나"$
- 평균 탐색 $\approx 1 + \alpha$: 거의 한 번에 찾지만, 꽉 차면 옆 칸을 기웃거려야 해서 α 만큼 조금씩 더 걸림
- 예: $\alpha = 0.5$ 면 평균 1.5번 봄, $\alpha = 0.9$ 면 평균 1.9번 봄 — 여유 공간이 있을수록 빠르다

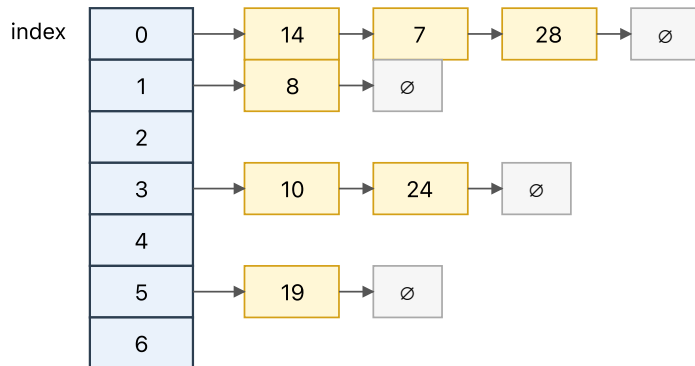
🔗 아이펜슬 필기 복원 (p.10)

- 해시 함수 $h(k) = k \bmod M$ 에서 M 은 소수 ★
- 적재율 $\alpha = n/m$, 평균 탐색 $\approx 1 + \alpha$
- 개선 흐름: 선형조사 → 클러스터링 → 이차조사 → secondary clustering → 이중해싱

체이닝 (개방 해싱)

체이닝 (Chaining / 개방 해싱): 같은 칸에 들어오면 연결리스트로 이어붙인다

해시함수 예: $h(k) = k \bmod 7$



평균 탐색 시간: $1 + \alpha$ ($\alpha = n/m$, 적재율). 해시함수만 잘 퍼뜨리면 사실상 $O(1)$

장점: 삽입 간단, 테이블 꽉 차도 동작. 단점: 포인터(연결리스트) 추가 공간.

충돌 시 같은 버킷에 연결 리스트로 이어 붙임

- 평균 탐색: $1 + \alpha$
- 장점: 삽입 간단, 테이블이 가득 차도 동작
- 단점: 포인터 추가 공간

선형 조사 (폐쇄 해싱)

선형 조사 (Linear Probing / 폐쇄 해싱): 충돌하면 "한 칸씩 옆으로"

$h(k) = k \bmod 7$, 삽입 순서: 7($\rightarrow 0$), 14($\rightarrow 0$, 충돌!), 8($\rightarrow 1$, 충돌!), 10($\rightarrow 3$), 24($\rightarrow 3$, 충돌!)

index	7	14	8	10	24	$\emptyset$	$\emptyset$
	0	1	2	3	4	5	6

시나리오: 14는 0번에 이미 7이 있어 $\rightarrow$ 1번으로, 1번도 점유면 $\rightarrow$ 2번... (한 칸씩 뒤로)

24는 3번에 10이 있어 $\rightarrow$ 4번에 자리 잡음

탐색 시에도 같은 규칙: $h(k)$ 에서 시작해 일치할 때까지 옆으로 이동, 빈 칸 만나면 "없음"으로 판정

문제점(클러스터링): 연속 점유 덩어리가 커질수록 탐색 비용이 커진다. $\rightarrow$ 이차조사/더블해싱으로 완화.

이차조사: $h(k)+1^2, +2^2, +3^2 \dots$ / 더블해싱: 점프 간격을 두 번째 해시함수로 결정.

충돌 시 한 칸씩 옆으로 빈 칸 찾기

$$h_i(k) = (h(k) + i) \bmod M$$

- 장점: 단순
- 단점: 클러스터링(연속 점유 덩어리가 커지면 탐색 비용 증가)

이차 조사

$$h_i(k) = (h(k) + i^2) \bmod M$$

일차 클러스터링 완화 $\rightarrow$ 하지만 **secondary clustering**이 남음

이중 해싱

$$h_i(k) = (h_1(k) + i \cdot h_2(k)) \bmod M$$

⚠ $h_2(k) \neq 0$ 필수 ★

🔗 해시 함수 종류 (PDF Ch6)

- 나눗셈법: $h(k) = k \bmod M$ — M 은 2의 거듭제곱에서 먼 소수
- 곱셈법: $h(k) = \lfloor M(kA \bmod 1) \rfloor$, $A \approx (\sqrt{5} - 1)/2$
- 중간제곱법: k^2 의 중간 비트 추출
- 폴딩법: 키를 여러 조각으로 쪼개 더함
- 좋은 해시함수 조건: ① 계산 빠름 ② 버킷에 균등 분포 ③ 충돌 최소화

🔗 해싱 탐색 성능 비교표 (PDF Ch6)

자료구조	평균 탐색 성공	평균 탐색 실패	최악
순차 탐색	$n/2$	n	n
이진 탐색	$\log n$	$\log n$	$\log n$
이진탐색트리(평균)	$\log n$	$\log n$	n (편향)
해싱(체이닝)	$1 + \alpha/2$	α (또는 $1 + \alpha$)	n
해싱(선형조사)	$\frac{1}{2}(1 + \frac{1}{1-\alpha})$	$\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$	n

$\alpha \rightarrow 1$ 에 가까울수록 선형조사 실패 비용 폭증 $\rightarrow \alpha < 0.7$ 권장

6-5. 선형 조사 손계산 (Pop Quiz 2 유형)

선형 조사 해싱: $h(k) = k \bmod 13$ 으로 키 [44, 37, 31, 63, 14, 58, 45, 18] 삽입
규칙: 자리가 차 있으면 오른쪽으로 한 칸씩 이동하며 빈 자리 찾기 (끝이면 처음으로 순환). 충돌이 길어질수록 성능 저하.

인덱스 ↓ 1) 44 삽입 → $h(44) = 44 \bmod 13 = 5$ (빈 자리) 8 9 10 11 12

					44							
--	--	--	--	--	----	--	--	--	--	--	--	--

2) 37 삽입 → $37 \bmod 13 = 11$ (빈 자리)

					44						37	
--	--	--	--	--	----	--	--	--	--	--	----	--

3) 31 삽입 → $31 \bmod 13 = 5$ (충돌!) → 오른쪽 한 칸 → 6 에 넣기

					44	31					37	
--	--	--	--	--	----	----	--	--	--	--	----	--

4) 63 삽입 → $63 \bmod 13 = 11$ (충돌!) → 12

					44	31					37	63
--	--	--	--	--	----	----	--	--	--	--	----	----

5) 14 삽입 → $14 \bmod 13 = 1$ (빈 자리)

14					44	31					37	63
----	--	--	--	--	----	----	--	--	--	--	----	----

6) 58 삽입 → $58 \bmod 13 = 6$ (충돌!) → 7

	14				44	31	58				37	63
--	----	--	--	--	----	----	----	--	--	--	----	----

7) 45 삽입 → $45 \bmod 13 = 6$ (충돌!) → 7 (또 충돌!) → 8

	14				44	31	58	45			37	63
--	----	--	--	--	----	----	----	----	--	--	----	----

8) 18 삽입 → $18 \bmod 13 = 5$ (충돌!) → 6 → 7 → 8 → 9 (네 번 미끄러진 끝에 도착) ★ 긴 덩어리 경고

	14				44	31	58	45	18		37	63
--	----	--	--	--	----	----	----	----	----	--	----	----

교훈(1차 군집화): 인접한 슬롯들이 덩어리로 뭉치면 새 키가 계속 미끄러진다. 이중해싱/이차조사로 완화 가능.

$h(k) = k \bmod 13$, 삽입 순서: 44, 37, 31, 63, 14, 58, 45, 18
최종 위치: 44→5, 37→11, 31→6, 63→12, 14→1, 58→7, 45→8, 18→9 (4번 미끄러짐)

7. 전략·정렬·탐색 비교표

7-1. 정렬 알고리즘

알고리즘	전략	최선	평균	최악	공간	안정성
선택	3장	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	불안정
삽입	4장	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	안정
병합	5장	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	안정
퀵	5장	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	스택	불안정
카운팅	6장	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	$O(n + k)$	안정
기수	6장	$\Theta(d(n + r))$	—	—	$O(n + r)$	안정

7-2. 탐색 알고리즘

알고리즘	조건	최선	평균	최악
순차	없음	$O(1)$	$O(n)$	$O(n)$

알고리즘	조건	최선	평균	최악
이진	정렬 필수	$O(1)$	$O(\log n)$	$O(\log n)$
해싱	좋은 해시	$O(1)$	$\approx O(1)$	충돌↑ 시 악화

7-3. 전략별 대비 (시험 단골)

쟁점	축소정복	분할정복	공간-시간
부분 문제 개수	1개	여러 개	해당 없음(테이블)
대표 예	이진 탐색, 삽입 정렬	병합/퀵, 스트라센	카운팅, 해싱, Horspool
추가 공간	적음	재귀 스택	많음 (의도적)
점화식	$T(n/2) + c$	$aT(n/b) + f(n)$	—

8. 필기 전용 포인트 (PDF 추출로는 빠지는 것)

슬라이드 위 아이펜슬 강조만 남은 포인트:

- 2장: $\alpha < \beta < \gamma$ 삼색 형광, $\log(n!)$ 양쪽 유도, "평균 복잡도는 입력 분포 가정 필요" 별주
- 3장: 0/1 배낭 2^n 숫자로 기입, TSP 대칭 시 $(n-1)!/2$ 주석
- 4장: 삽입 정렬 최선 $O(n)$ 에 "이동 없음" 별표, Quick Select 3경우 번호, 거듭제곱 짝·홀 식 밑줄
- 5장: 스트라센 $P_1 \sim P_7$ 메모, 최근접 쌍 strip 너비 $= d$, 피보나치 재귀 중복 노드 빨간 동그라미
- 6장: 카운팅 안정성 → 기수 정렬 연결 화살표, Horspool shift 규칙, 해싱 M 은 소수 ★, 이중 해싱 $h_2(k) \neq 0$ 경고

9. 시험 직전 체크리스트

✓ 아래를 말로 설명할 수 있으면 준비 완료

- ☐ O, Ω, Θ 정의와 "상한·하한·정확한 차수" 비유
- ☐ 복잡도 위계 11단 (1부터 n^n 까지)
- ☐ 선택 정렬은 입력 상태 무관 $\Theta(n^2)$
- ☐ 삽입 정렬은 거의 정렬 시 $O(n)$
- ☐ 이진 탐색의 전제: 정렬 필수
- ☐ 병합 vs 퀵: 병합은 항상 $n \log n$, 퀵은 최악 n^2
- ☐ DFS/BFS 모두 $O(V + E)$ (인접 리스트)
- ☐ 위상 정렬 전제: DAG · 결과 복수 가능
- ☐ 분할정복 점근 복잡도 결과 암기 (마스터 정리 유도 X)
- ☐ 스트라센: 곱셈 $8 \rightarrow 7$ 회, $O(n^{2.807})$
- ☐ Horspool: 비교 오른쪽→왼쪽, 이동은 텍스트 문자 기준
- ☐ 해싱 평균 $= 1 + \alpha$, 선형조사 약점 클러스터링
- ☐ 이중 해싱 $h_2(k) \neq 0$

10. 핵심 암기 카드 (30초 암송)

숫자·공식 총정리

- 점화식 → 복잡도: $T(n/2) + 1 \rightarrow \log n \cdot 2T(n/2) + n \rightarrow n \log n \cdot 2T(n-1) + 1 \rightarrow 2^n$
- 이진 탐색: 전체 정렬 필수 · $O(\log n)$
- 삽입 정렬 최선: $O(n)$ (이동 없음)
- 선택 정렬: 항상 $\Theta(n^2)$
- 병합 vs 퀵: 병합 항상 $n \log n$ / 퀵 최악 n^2
- **Quick Select**: 평균 $O(n)$, 최악 $O(n^2)$
- **DFS/BFS**: 인접 리스트 $O(V + E)$ / 인접 행렬 $O(V^2)$
- 위상 정렬: **DAG** 필요, 결과 복수
- 분할정복 접근 (결과만): 병합·퀵평균 $n \log n$ / 이진탐색·빠른거듭제곱 $\log n$ / 스트라센 $n^{2.807}$
- 스트라센: 곱셈 7회, $O(n^{2.807})$
- **Horspool shift**: 마지막 문자 제외, 패턴에 없으면 m
- 해싱: M 소수 · $\alpha = n/m$ · 평균 $1 + \alpha$
- 이중 해싱: $h_2(k) \neq 0$

11. 한 줄 결론

"이 과목은 알고리즘 이름 암기가 아니라, 문제를 전략으로 분류하고 왜 빠르거나 느린지 설명하는 훈련이다."

부록. 본 정리에 임베드된 SVG 28종

#	주제	파일
1	점근 표기 $O/\Omega/\Theta$	big-o-omega-theta.svg
2	복잡도 위계	complexity-hierarchy.svg
3	선택 정렬	selection-sort.svg
4	삽입 정렬	insertion-sort.svg
5	이진 탐색	binary-search.svg
6	병합 정렬	merge-sort.svg
7	퀵 정렬 분할 (1회)	quick-sort-partition.svg
7-1	퀵 정렬 전체 과정 (PDF 예시)	quick-sort-full.svg
8	카운팅 정렬	counting-sort.svg
9	Horspool	horspool.svg
10	해시 체이닝	hash-chaining.svg
11	해시 선형 조사	hash-linear-probing.svg
12	DFS vs BFS (트리)	dfs-bfs.svg
12-1	DFS / BFS 그래프 풀이 (9-node)	dfs-bfs-graph.svg
13	0/1 배낭 완전탐색	knapsack-bruteforce.svg
14	위상 정렬 단계	topological-sort-steps.svg
14-1	DFS 위상 정렬 타임라인	dfs-topological-trace.svg
15	빠른 거듭제곱	fast-exponentiation.svg
16	Quick Select	quick-select.svg
17	피보나치 재귀 트리	fibonacci-recursion-tree.svg
18	선형 조사 삽입 과정	linear-probing-insert.svg

#	주제	파일
19	유클리드 호제법 (GCD)	gcd-euclidean.svg
20	순차 탐색	sequential-search.svg
21	TSP 완전탐색	tsp-bruteforce.svg
22	최근접 쌍 예시 풀이 (PDF 6점)	closest-pair-example.svg
23	기수 정렬	radix-sort.svg
24	Boyer-Moore 원형	boyer-moore.svg
25	스트라센 행렬 곱셈 예시 ($M_1 \sim M_7$)	strassen-example.svg